

EdgeYOLO-RS: A Lightweight Rotation-Aware Object Detection Framework for Remote Sensing Imagery Using YOLOv8-OBB

*A Project Report submitted in the partial fulfillment of
the Requirements for the award of the degree*

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING
Submitted by

B.Uma Bhavani (22471A0563)

Under the esteemed guidance of

Dr.S. Siva Nageswara Rao .B.Tech, M.Tech., Ph.D.

Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

NARASARAOPETA ENGINEERING COLLEGE: NARASAROPET
(AUTONOMOUS)

Accredited by NAAC with A+ Grade and NBA under Tyre -1 NIRF rank
in the band of 201-300 and an ISO 9001:2015 Certified

Approved by AICTE, New Delhi, Permanently Affiliated to JNTUK, Kakinada

KOTAPPAKONDA ROAD, YALAMANDA VILLAGE, NARASARAOPET- 522601

2025-2026

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project that is entitled with the name **“EdgeYOLO- RS: A Lightweight Rotation-Aware Object Detection Framework for Remote Sensing Imagery Using YOLOv8-OBB”** is a bonafide work done by **B.Uma Bhavani(22471A0563)** in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of COMPUTER SCIENCE AND ENGINEERING during 2025-2026.

PROJECT GUIDE

Dr.S.Siva Nagheswara Rao, B.Tech., M.Tech., Ph.D.
Professor

PROJECT CO-ORDINATOR

Dr.Sireesha Moturi, B.Tech., M.Tech., Ph.D.
Associate Professor

HEAD OF THE DEPARTMENT

Dr. S. N. Tirumala Rao, M.Tech., Ph.D.
Professor & HOD

EXTERNAL EXAMINER

DECLARATION

I declare that this project work titled "**EdgeYOLO-RS: A Lightweight Rotation-Aware Object Detection Framework for Remote Sensing Imagery Using YOLOv8-ORB**" is composed by me that the work contain here is my own except where explicitly stated otherwise in the text and that this work has been submitted for any other degree or professional qualification except as specified.

B.Uma Bhavani(22471A0563)

ACKNOWLEDGEMENT

I wish to express my thanks to various personalities who are responsible for the completion of my project. I am extremely thankful to my beloved chairman, **Sri M. V. Koteswara Rao, B.Sc.**, who took keen interest in me in every effort throughout this course. I owe my sincere gratitude to my beloved principal, **Dr. S. Venkateswarlu, Ph.D.**, for showing his kind attention and valuable guidance throughout the course.

I express my deep-felt gratitude towards **Dr. S. N. Tirumala Rao, M.Tech., Ph.D.**, HOD of the CSE department, and also to my guide, **Dr. S.Siva Nageswara Rao B.Tech., M.Tech., Ph.D.**, Professor of the CSE department, whose valuable guidance and unstinting encouragement enabled me to accomplish my project successfully in time.

I extend my sincere thanks to **Dr. Moturi Sireesha, B.Tech., M.Tech., Ph.D.** Associate Professor & Project Coordinator of the project, for extending his encouragement. Their profound knowledge and willingness have been a constant source of inspiration for me throughout this project work.

I extend my sincere thanks to all the other teaching and non-teaching staff in the department for their cooperation and encouragement during my B.Tech. degree.

I have no words to acknowledge the warm affection, constant inspiration, and encouragement that I received from my parents.

I affectionately acknowledge the encouragement received from my friends and those who were involved in giving valuable suggestions and clarifying my doubts, which really helped me in successfully completing my project.

By

B.Uma Bhavani (22471A0563)



INSTITUTE VISION AND MISSION

INSTITUTION VISION

To emerge as a Centre of excellence in technical education with a blend of effective student centric teaching learning practices as well as research for the transformation of lives and community.

INSTITUTION MISSION

M1: Provide the best class infra-structure to explore the field of engineering and research

M2: Build a passionate and a determined team of faculty with student centric teaching, imbining experiential, innovative skills

M3: Imbibe lifelong learning skills, entrepreneurial skills and ethical values in students for addressing societal problems



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VISION OF THE DEPARTMENT

To become a center of excellence in nurturing the quality Computer Science & Engineering professionals embedded with software knowledge, aptitude for research and ethical values to cater to the needs of industry and society.

MISSION OF THE DEPARTMENT

The department of Computer Science and Engineering is committed to

M1: Mould the students to become Software Professionals, Researchers and Entrepreneurs by providing advanced laboratories.

M2: Impart high quality professional training to get expertise in modern software tools and technologies to cater to the real time requirements of the Industry.

M3: Inculcate team work and lifelong learning among students with a sense of societal and ethical responsibilities.



Program Specific Outcomes (PSO's)

PSO1: Apply mathematical and scientific skills in numerous areas of Computer Science and Engineering to design and develop software-based systems.

PSO2: Acquaint module knowledge on emerging trends of the modern era in Computer Science and Engineering

PSO3: Promote novel applications that meet the needs of entrepreneur, environmental and social issues.

Program Educational Objectives (PEO's)

The graduates of the programme are able to:

PEO1: Apply the knowledge of Mathematics, Science and Engineering fundamentals to identify and solve Computer Science and Engineering problems.

PEO2: Use various software tools and technologies to solve problems related to the academia, industry and society.

PEO3: Work with ethical and moral values in the multi-disciplinary teams and can communicate effectively among team members with continuous learning.

PEO4: Pursue higher studies and develop their career in software industry.

Program Outcomes

PO1: Engineering knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4).

PO3: Design/development of solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5).

PO4: Conduct investigations of complex problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering tool usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9).

PO8: Individual and Collaborative teamwork: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project management and finance: Apply knowledge and understanding of engineering management principles and economic decisionmaking and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Project Course Outcomes (CO'S):

CO421.1: Analyse the System of Examinations and identify the problem.

CO421.2: Identify and classify the requirements.

CO421.3: Review the Related Literature

CO421.4: Design and Modularize the project

CO421.5: Construct, Integrate, Test and Implement the Project.

CO421.6: Prepare the project Documentation and present the Report using appropriate method.

Course Outcomes – Program Outcomes mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
C421.1		✓										✓		
C421.2	✓		✓		✓							✓		
C421.3				✓		✓	✓	✓				✓		
C421.4			✓			✓	✓	✓				✓	✓	
C421.5					✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
C421.6									✓	✓	✓	✓	✓	

Course Outcomes – Program Outcome correlation

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
C421.1	2	3										2		
C421.2			2		3							2		
C421.3				2		2	3	3				2		
C421.4			2			1	1	2				3	2	
C421.5					3	3	3	2	3	2	2	3	2	1
C421.6									3	2	1	2	3	

Note: The values in the above table represent the level of correlation between CO's and PO's:

1. Low level
2. Medium level
3. High level

Project mapping with various courses of Curriculum with Attained PO's:

Name of the course from which principles are applied in this project	Description of the device	Attained PO
C2204.2, C22L3.2	Gathering the requirements and defining the problem, plan to develop model for detection and classification of objects in remote sensing images using YOLOv8-OB	PO1, PO3,PO8
CC421.1, C2204.3, C22L3.2	Each and every requirement is critically analyzed, the process model is identified	PO2, PO3, PO8
CC421.2, C2204.2, C22L3.3	Logical design is done by using the unified modelling language which involves individual team work	PO3, PO5, PO8, PO9
CC421.3, C2204.3, C22L3.2	Each and every module is tested, integrated, and evaluated in our project	PO1, PO5, PO8
CC421.4, C2204.4, C22L3.2	Documentation is done by all our three members in the form of a group	PO8,PO10
CC421.5, C2204.2, C22L3.3	Each and every phase of the work in group is presented periodically	PO8,PO10, PO11
C2202.2, C2203.3, C1206.3, C3204.3, C4110.2	Implementation is done and the project will be handled by the social media users and in future updates in our project can be done based on detection for remote sensing objects	PO4, PO7, PO8
C32SC4.3	The physical design includes website to detect objects in the remote sensing images	PO5, PO6, PO8

ABSTRACT

Object detection in remote sensing images is particularly challenging due to the arbitrary orientations of objects, variations in scale, and dense scene layouts. This work presents EdgeYOLO-RS, a lightweight detection framework specifically designed for aerial imagery. Based on the YOLOv8-OBB architecture, the model leverages oriented bounding box regression to accurately localize rotated entities such as ships, vehicles, and infrastructure elements. The system is trained on the DOTA v2.0 dataset, a widely-used benchmark for remote sensing, with customized preprocessing including label normalization, image sharpening, and uniform resolution scaling to enhance detection outcomes.

The training pipeline uses 1024×1024 contrast-enhanced images and is optimized for efficient deployment on edge devices. Both visual results and evaluation metrics demonstrate that EdgeYOLO-RS performs reliably in real-world contexts such as UAV monitoring, emergency response, and geospatial mapping. The framework also lays the groundwork for future enhancements involving attention mechanisms and videobased remote sensing analysis.

INDEX

S.NO	CONTENT	PAGE NO
1	INTRODUCTION	1
	1.1 MOTIVATION	4
	1.2 PROBLEM STATEMENT	5
	1.3 OBJECTIVE	6
2	LITERATURE SURVEY	7
3	SYSTEM ANALYSIS	
	3.1 EXISTING SYSTEM	12
	3.1.1 DISADVANTAGES OF THE EXISTING SYSTEM	15
	3.2 PROPOSED SYSTEM	17
	3.3 FEASIBILITY STUDY	18
	3.4 USING COCOMO MODEL	20
4	SYSTEM REQUIREMENTS	
	4.1 SOFTWARE REQUIREMENTS	21
	4.2 REQUIREMENT ANALYSIS	21
	4.3 HARDWARE REQUIREMENTS	22
	4.4 SOFTWARE	22
	4.5 SOFTWARE DESCRIPTION	23
5	SYSTEM DESIGN	
	5.1 SYSTEM ARCHITECTURE	24
	5.1.1 DATASET	27
	5.1.2 DATA PREPROCESSING	28
	5.1.3 FEATURE EXTRACTION	29
	5.1.4 MODEL BUILDING	30
	5.1.5 CLASSIFICATION	35
	5.2 MODULES	39
	5.3 UML DIAGRAMS	42
6	IMPLEMENTATION	
	6.1 MODEL IMPLEMENTATION	46
	6.2 CODING	49
7	TESTING	

7.1	UNIT TESTING	56
7.2	INTEGRATION TESTING	57
7.3	SYSTEM TESTING	60
8	RESULT ANALYSIS	65
9	OUTPUT SCREENS	72
10	CONCLUSION	74
11	FUTURE SCOPE	75
12	REFERENCES	77

LIST OF FIGURES

S.NO	FIGURE DESCRIPTION	PAGE NO
1	FIG 1.1 ORIENTED OBJECT DETECTION IN AERIAL IMAGER USING EDGE-YOLO-RS	3
2	FIG 3.1 FLOW CHART OF EXISTING SYSTEM FOR REMOTE SENSING OBJECT DETECTION	13
3	FIG 3.2 FLOWCHART OF PROPOSED SYSTEM	16
4	FIG 5.1 DIFFERENT DATA SET IMAGES	27
5	FIG 5.2 OUTPUT SHOWING OBB-BASED DETECTION USING YOLO8-OBB	29
6	FIG 5.3 YOLOv8 - OBB ARCHITECTURE	32
7	FIG 5.4 YOLOv8 – OBB MODEL AND OBB - NMS	33
8	FIG 5.5 CLASSIFICATION DIFFERENT OBJECTS	36
9	FIG 5.6 PROCESS FLOWCHART	43
10	FIG 5.7 UML DIAGRAM FOR AERIAL OBJECT DETECTION USING YOLOv8 - OBB	44
11	FIG 7.1 VEHICLES DETECTED	62
12	FIG 7.2 STATUS NOT DETECTED	63
13	FIG 8.1 PRECISION AND RECALL OVER EPOCHS	65
14	FIG 8.2 YOLOv8 TRAINING METRICS	66
15	FIG 8.3 CLASS DISTRIBUTION IN TRAINING SET	67
16	FIG 8.4 <u>MAP@0.5</u> COMPARISON ACROSS MODELS	68
17	FIG 8.5 PRECISION COMPARISON ACROSS MODELS	68
18	FIG 8.6 RECALL COMPARISON ACROSS MODELS	68
19	FIG 8.7 CONFUSION MATRIX FOR YOLO – OBB MODEL	69
20	FIG 8.8 VEHICLE DETECTION OUTPUT	70
21	FIG 8.9 NO OBJECT DETECTED CASE	70
22	FIG 8.10 SHIP DETECTION OUTPUT	70
23	FIG 9.1 HOME PAGE	72

List of Tables

S.NO	CONTENT	PAGE NO
1	TABLE 1. DATASET DESCRIPTION	26
2	TABLE 2. OVERALL PERFORMANCE SUMMARY	71

1.INTRODUCTION

Object detection has become one of the most transformative areas in the field of computer vision, enabling machines to identify and locate objects within digital images or videos. This technology serves as the foundation for numerous real-world applications, ranging from autonomous vehicles and surveillance systems to environmental monitoring and military reconnaissance. However, while conventional object detection systems have achieved remarkable success on natural or ground-level images, detecting objects in **remote sensing imagery**—such as aerial or satellite views—presents a unique set of challenges that demand specialized solutions.

Remote sensing images are typically captured from elevated platforms like satellites, aircraft, and Unmanned Aerial Vehicles (UAVs). These images provide crucial information for **urban planning, disaster management, environmental protection, and national security**. The objects within such images, however, differ significantly from those found in everyday photographs. They often appear at arbitrary orientations, varying scales, and within densely populated or cluttered scenes. Traditional detectors, which rely on **axis-aligned bounding boxes**, often struggle in these scenarios because they fail to capture the rotational and spatial variations inherent in aerial imagery.

The increasing availability of **high-resolution aerial imagery** has led to an exponential rise in the amount of data being collected daily. Processing this vast amount of visual information manually is both time-consuming and error-prone. Moreover, many aerial monitoring systems are deployed on **edge devices**—such as drones, portable surveillance systems, or mobile mapping platforms—where computational resources are limited. Hence, there is an urgent need for **lightweight, rotation-aware, and real-time object detection frameworks** that can operate efficiently in such constrained environments while maintaining high accuracy.

Existing deep learning–based detectors, including **Faster R-CNN, SSD, and YOLO**, have demonstrated impressive results in general-purpose detection tasks. Among them, the **You Only Look Once (YOLO)** family of models is known for its balance of speed and accuracy, making it particularly suitable for real-time detection. However, conventional YOLO variants are primarily designed for

detecting objects in horizontal or upright positions, which limits their performance when applied to aerial images where objects—such as airplanes, ships, vehicles, and buildings—are oriented at various angles. To address this limitation, researchers have explored **Oriented Object Detection (OBB)**, where bounding boxes are rotated to align with the object’s true orientation. This approach significantly improves localization accuracy and reduces background noise in detection results. One of the most widely used benchmarks for such tasks is the **DOTA (Dataset for Object Detection in Aerial Images)**, which includes multi-class, multi-angle object annotations suitable for training rotation-aware models.

The proposed system, **EdgeYOLO-RS**, builds upon this foundation by extending the latest **YOLOv8-OBB** architecture. This model integrates rotation-invariant bounding box regression with a compact and efficient neural backbone, enabling accurate detection even in complex, high-density aerial scenes. It has been specifically designed for **real-time edge deployment**, meaning it can function effectively on devices with limited processing power while maintaining detection accuracy comparable to larger, resource-intensive models.

To ensure optimal performance, the EdgeYOLO-RS framework utilizes the **DOTA v2.0 dataset**, enhanced through a series of preprocessing steps including **resolution normalization, adaptive contrast enhancement, image sharpening, and label verification**. The model was trained using high-resolution (1024×1024) images and fine-tuned with data augmentations such as random flipping, mosaic transformation, and HSV adjustment. The training was conducted using **Google Colab with GPU acceleration**, leveraging the Ultralytics YOLOv8-OBB API for efficient experimentation.

The significance of this work lies in its **practical applicability to real-world problems**. EdgeYOLO-RS can be deployed in a variety of remote sensing applications, such as:

- **UAV-based surveillance and reconnaissance**, enabling real-time detection of vehicles or infrastructure.
- **Disaster management**, for identifying damaged structures or monitoring flood-affected areas.

- **Maritime monitoring**, for ship detection and tracking.
- **Urban and environmental mapping**, facilitating intelligent city planning and resource management.

by achieving a balance between **accuracy, speed, and resource efficiency**, EdgeYOLO-RS establishes a new benchmark for lightweight aerial object detection systems..

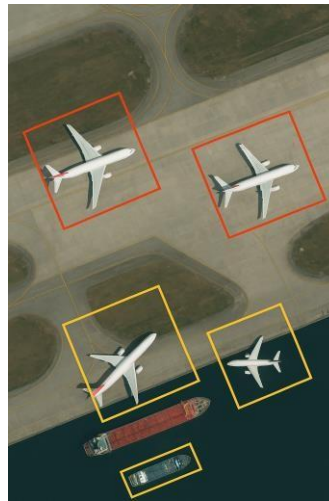


FIG 1. 1 ORIENTED OBJECT DETECTION IN AERIAL IMAGER USING EDGE-YOLO-RS

Fig 1.1 illustrates oriented object detection in aerial imagery using the proposed EdgeYOLO-RS framework. The image shows an airport captured from a satellite view, where multiple airplanes and ships are enclosed within rotated bounding boxes that align precisely with each object's orientation. This visualization demonstrates the importance of rotation-aware detection, as traditional axis-aligned boxes often fail to capture the true shape and angle of aerial targets. By utilizing oriented bounding boxes (OBBs), the EdgeYOLO- RS model achieves more accurate localization and reduced background interference, making it highly effective for real-world remote sensing tasks such as UAV surveillance and geospatial monitoring.

1.2 Motivation

Object detection in remote sensing imagery has become an essential component of modern geospatial intelligence, with applications spanning **urban planning, disaster management, defense surveillance, environmental monitoring, and infrastructure mapping**. The ability to automatically detect and identify objects such as airplanes, ships, vehicles, and buildings from aerial or satellite images can provide critical insights for decision-making in both civilian and defense sectors. However, traditional object detection models, designed primarily for ground-level natural images, often fail to perform effectively when applied to aerial perspectives.

One of the major challenges in remote sensing is the **arbitrary orientation** of objects. Unlike natural images where objects are typically upright, aerial images capture scenes from top-down views, resulting in objects appearing at various angles, scales, and densities. This leads to frequent misdetections or poor localization when using conventional **axis-aligned bounding boxes**, which are unable to represent the true orientation of targets. Moreover, the high variability in object sizes, lighting conditions, and environmental clutter further complicates detection accuracy.

In addition, most high-performing detection models require **powerful GPUs and large computational resources**, making them difficult to deploy on **edge devices** such as drones and field monitoring systems. This limits their practical use in real-time scenarios, particularly in environments where quick responses are crucial — for example, during emergency rescue operations, forest fire monitoring, or maritime surveillance. Ultimately, this project seeks to provide a **lightweight, real-time, and rotation-aware object detection system** that can be deployed in practical remote sensing applications. By optimizing detection for edge devices, EdgeYOLO-RS aims to make advanced aerial analytics more accessible, reduce reliance on server-grade systems, and contribute to smarter, faster, and more reliable geospatial monitoring.

1.3 Problem Statement

The rapid growth of aerial and satellite imaging technologies has resulted in massive volumes of visual data, offering valuable insights for **urban planning, environmental monitoring, defense surveillance, and disaster management**. However, detecting objects accurately from such images remains a major challenge due to their **high variability in orientation, scale, and density**. Unlike ground-level images, aerial objects such as airplanes, ships, and vehicles appear rotated or skewed, making traditional **axis-aligned bounding box detectors** inadequate for precise localization.

Moreover, most advanced deep learning models are designed for **high-end computational setups**, limiting their deployment on **edge devices** like drones and portable sensors. This creates a gap between high-accuracy models and real-time applications in field environments where quick decisions are critical.

Another issue is the **lack of rotation-aware and lightweight detection frameworks** optimized for remote sensing imagery. Existing models that incorporate orientation awareness, such as transformer-based or hybrid networks, often suffer from excessive model size and high computational costs, making them unsuitable for real-time edge deployment. As a result, there is a pressing need for a **compact, rotation-invariant, and edge-optimized detection system** that can maintain high accuracy without compromising on speed or efficiency.

To overcome these limitations, there is a need for a **lightweight, rotation-aware, and efficient object detection framework** capable of performing in real-time under limited computational resources. The proposed **EdgeYOLO-RS** model addresses this by integrating **oriented bounding box (OBB) regression** into the **YOLOv8 architecture**, enabling accurate detection of rotated objects while maintaining high speed and compactness. This approach aims to achieve a balance between **detection precision and computational efficiency**, making it suitable for **UAVs, remote sensing platforms, and edge-based monitoring systems**.

1.4 Objective

The primary objective of this project is to develop a lightweight and rotation-aware object detection framework that can efficiently identify and localize objects in aerial imagery. The proposed EdgeYOLO-RS model aims to overcome the limitations of conventional object detectors by incorporating oriented bounding box regression into the YOLOv8 architecture, enabling accurate detection of rotated objects that appear at different angles and scales.

This project focuses on achieving high accuracy and real-time performance while maintaining a compact model suitable for deployment on edge devices such as drones and mobile sensors. To accomplish this, the system emphasizes data preprocessing, including normalization, sharpening, and contrast enhancement of aerial images from the DOTA v2.0 dataset, ensuring better feature extraction and model learning.

Another key objective is to train and validate the model on high-resolution images to ensure consistent performance across various object categories and environmental conditions. The framework is optimized to deliver fast inference speeds with minimal computational requirements, making it practical for real-world applications.

The main objective of this project is to develop a lightweight and rotation-aware object detection framework, EdgeYOLO-RS, capable of accurately identifying and localizing objects in aerial imagery. By integrating oriented bounding box regression into the YOLOv8 architecture, the model aims to detect rotated objects more effectively while maintaining high speed and efficiency.

Finally, the project seeks to evaluate the model's performance using precision, recall, and mean average precision (mAP) metrics while comparing it with existing axis-aligned detection systems. By achieving this balance between detection accuracy and computational efficiency, EdgeYOLO-RS aims to provide a reliable solution for real-time aerial surveillance, environmental monitoring, and other remote sensing applications.

2. LITERATURE SURVEY

Object detection in remote sensing imagery has been one of the most widely researched areas in computer vision due to its applications in urban planning, disaster response, military surveillance, maritime monitoring, and environmental analysis. Traditional computer vision methods such as Histogram of Oriented Gradients (HOG), Haar cascades, Support Vector Machines (SVM), and Random Forests were initially used to detect objects in aerial images. However, these classical techniques suffer from severe limitations, including poor generalization, inability to detect rotated objects, and dependency on handcrafted features, making them unsuitable for complex remote-sensing environments.

With the evolution of Deep Learning, Convolutional Neural Networks (CNNs) revolutionized object detection by enabling automatic feature extraction and significantly improving accuracy. Modern deep detectors like Faster R-CNN, SSD, and the YOLO series have been successfully adapted for aerial imagery. However, most conventional models rely on axis-aligned bounding boxes, which fail in remote-sensing scenarios where objects appear in arbitrary orientations, exhibit large scale differences, or are densely packed. To overcome these limitations, researchers shifted toward **Oriented Bounding Box (OBB)** detection, which allows bounding boxes to rotate according to the object's orientation, improving localization and reducing background noise.

Huang et al. [3] introduced LO-Det, a lightweight oriented detection model using Channel Separation Aggregation (CSA), Dynamic Receptive Fields (DRF), and a Diagonal Support Constraint Head. Their architecture improved stability in OBB regression while maintaining high inference speed. The model proved that lightweight architectures can achieve competitive performance on datasets like DOTA by optimizing feature fusion and reducing computational

Zhang et al. [4] proposed an efficient oriented YOLOv8 variant with enhancements such as Wavelet-based C2f modules, Adaptive Scale Feature Pyramid (ASFP), and GhostDynamicConv. This configuration improved small- object recognition and reduced model size to 21.6M parameters, demonstrating the evolving trend toward compact, rotation-aware detectors suitable for real-time aerial inference.

Ju et al. [5] developed a hybrid attention YOLOv8-OBB model that integrates Mixed Local Channel Attention (MLCA), dynamic upsampling, and a RepNCSP-ELAN module. Their model reduced network complexity and improved mAP on DOTA by ~2.3%, highlighting the impact of attention-based lightweight refinement on OBB performance.

Yu et al. [6] introduced CIMB-YOLOv8, a multi-branch contextual fusion detector incorporating space-to-depth modules and contextual attention mechanisms. Their work achieved 68.1% mAP on DOTA with impressive FPS, proving that context-aware designs significantly aid in detecting small, cluttered targets in aerial scenes.

Another noteworthy method is the Faster R-CNN [11], which employs region proposal networks for real-time tumor detection in MRI images. This technique stands out for its high precision, enabling rapid and accurate tumor identification while reducing the time required for diagnosis in the imaging process.

Tong et al. [8] proposed YOLO-Faster, a reduced and optimized variant of YOLOv5 designed specifically for small oriented targets. Their lightweight improvements demonstrated a strong balance between speed and accuracy, reinforcing the importance of model simplification for edge deployments.

Gao et al. [7] focused on oriented ship detection in SAR imagery with R-LRBPNet. Their model employed bottleneck transformer modules, angle-decoupled regression, and ProbIoU loss, providing robust detection in densely packed maritime environments. This work is particularly relevant for scenarios where objects overlap heavily or exhibit extreme orientations.

Shi et al. [9] extended YOLOv8 with wavelet-based feature pyramids and adaptive feature fusion for improved detection in multi-scale aerial scenarios. Their work strengthened rotation-invariant detection by preserving fine-grained spatial features across layers.

Ahmad et al. [10] studied dynamic smooth feature fusion for oriented detection, optimizing the trade-off between computational cost and accuracy, especially for remote-sensing imagery. Their findings emphasize the impact of efficient feature aggregation in rotation-aware tasks.

Li et al. [11] optimized YOLOv8 for ship detection through EMA attention, image-scale augmentations, and BiFPN-based multi-scale fusion. Their improvements demonstrated how data augmentation and enhanced feature pyramids contribute to significant performance gains in aerial detection pipelines.

Xie et al. [12] surveyed small-object detection challenges in aerial images, outlining constraints such as rotation, scale variations, occlusion, and dense distributions. Their survey summarized strategies like OBB regression, multiscale fusion, and attention mechanisms, laying the conceptual foundation for modern frameworks like EdgeYOLO-RS.

In addition to YOLO-based methods, transformer-driven architectures such as OrientedFormer [18] have also emerged. These models incorporate self-attention and rotation-aware positional encoding, enabling precise detection in complex scenes. Although extremely powerful, such transformer-based models generally require high computational resources, making them less suitable for low-power edge devices.

The DOTA dataset introduced by Ding et al. [20] remains the most widely adopted benchmark in oriented aerial object detection. With high-resolution images and 18 object categories annotated using quadrilateral OBBs, DOTA catalyzed the development of modern oriented detectors like YOLOv8-OBB and its improved variants. The scale diversity, clutter, and extreme orientations in DOTA emphasize the need for rotation-invariant and lightweight models.

Building upon these advancements, the proposed **EdgeYOLO-RS** aligns closely with the lightweight and rotation-aware direction seen in contemporary research. Unlike heavy transformer models, EdgeYOLO-RS leverages the native YOLOv8-OBB architecture and introduces custom preprocessing, label validation, sharpening filters, and uniform resolution normalization to enhance performance on DOTA. The model maintains a compact footprint (<10 MB), enabling deployment on UAVs, drones, and edge-AI devices where computational resources are limited.

Furthermore, the integration of adaptive contrast enhancement, rotation-based augmentations, and high-resolution (1024×1024) training contributes to improved recognition of small and rotated objects. The framework’s performance reflects the benefits of lightweight OBB-aware detectors — achieving strong mAP, precision, and recall while supporting real-time inference.

Deep Learning continues to dominate remote-sensing detection research due to its ability to automatically extract spatial, rotational, and contextual patterns. Architectures like CNNs, transformers, and attention-based fusion networks provide robust feature understanding across modalities and resolutions. Transfer learning, data augmentation, and model compression further enhance performance, especially in scenarios involving limited computational power or imbalanced datasets common in aerial imaging.

Recent research emphasizes that rotation-invariant detection is no longer optional but essential for accurate recognition in aerial and satellite imagery. Traditional horizontal bounding box models often misinterpret long, thin structures such as bridges, runways, and ships, especially when these objects appear at arbitrary angles. To address this limitation, advanced OBB-based detectors integrate angle regression and polygonal annotation formats that closely match the true geometry of aerial objects. Models like Rotated R-CNN, Oriented R-CNN, and ReDet introduced rotation-equivariant convolutions and rotated RoI pooling techniques that significantly enhance detection accuracy in cluttered scenes. These innovations deeply influenced later YOLO-based OBB architectures, proving that orientation-aware modules are critical for reliable remote-sensing object detection.

Recent literature also explores the integration of **multimodal data sources**, including optical imagery, SAR (Synthetic Aperture Radar), thermal imaging, and multispectral data, to improve detection reliability in challenging environments. Multimodal fusion helps detectors manage weather variations, nighttime imagery, occlusions, and low-contrast scenarios common in aerial surveillance. High-resolution datasets like DOTA, HRSC2016, and FAIR1M have driven advancements in feature-extraction networks capable of processing extremely fine spatial details. Models enhanced with attention mechanisms, multi-scale pyramids, and transformer-based encoders have demonstrated superior performance on these complex datasets. These multimodal and high-resolution trends highlight the growing need for flexible architectures that can generalize across diverse sensing conditions—further reinforcing the relevance of edge-optimized, orientation-aware detectors.

In summary, the literature demonstrates an ongoing shift toward **lightweight, rotation-aware, and edge-deployable** detectors. The proposed EdgeYOLO-RS framework follows this trajectory by combining YOLOv8-OBB’s accuracy with a streamlined training pipeline, making it a practical and efficient solution for real-world aerial object detection.

3. SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

Traditional object detection systems used in remote sensing environments have largely relied on classical computer vision techniques and axis-aligned bounding box (HBB) detectors. These methods, although foundational, face significant challenges when applied to aerial imagery captured by satellites, drones, and UAVs. Earlier detection models were not designed to handle the complexity of remote-sensing scenes, where objects appear at arbitrary orientations, vary greatly in scale, and often exist in dense clusters. As a result, conventional systems frequently misidentify targets, produce inaccurate bounding boxes, or fail entirely when objects rotate or overlap.

Classical Machine Learning approaches such as Support Vector Machines (SVM), AdaBoost, Random Forests (RF), and k-Means clustering were historically used for tasks like ship detection, vehicle counting, or building identification. These systems depended heavily on handcrafted features — including texture descriptors, gradient-based features (HOG/SIFT), morphological cues, and spectral signatures. Although useful for simple datasets, these methods show poor generalization when applied to large-scale aerial datasets like DOTA. Their accuracy typically ranged between 50–70%, highly dependent on feature quality and preprocessing. Additionally, axis-aligned bounding box detectors struggle in aerial scenarios where most objects (ships, aircraft, bridges, or vehicles) are naturally rotated at various angles, causing significant localization errors.

To address accuracy limitations, researchers adopted rotation-aware detectors like Rotated R-CNN, R-YOLO, and RoI Transformer. While these models improved detection of arbitrarily oriented objects, they introduced added complexity, slower inference times, and heavier architectures unsuitable for real-time edge deployment. These limitations in speed, scalability, and computational cost created the need for a more efficient, lightweight, and orientation-aware system like EdgeYOLO-RS.

FIG 3.1 FLOW CHART OF EXISTING SYSTEM FOR REMOTE-SENSING OBJECT DETECTION

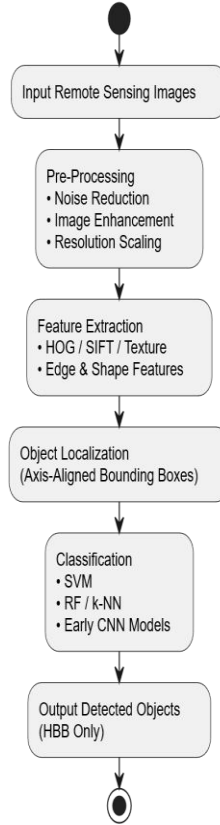


FIG 3.1. FLOW CHART OF EXISTING SYSTEM FOR REMOTE-SENSING OBJECT DETECTION

The existing system for remote-sensing object detection begins with an input dataset containing high-resolution satellite or aerial images. These images undergo a pre-processing phase to correct lighting variations, reduce atmospheric noise, and normalize resolution. Traditional enhancement techniques — such as histogram equalization, sharpening filters, and Gaussian smoothing — attempt to improve image clarity. However, these classical methods often fail in scenes containing low contrast, cloud cover, or background clutter.

Feature extraction relies heavily on handcrafted descriptors such as histogram-based features, gradient patterns, edge maps, and shape signatures. These manually engineered features provide limited representation capability, making it difficult to capture complex object structures or small-scale targets commonly present in datasets like DOTA.

Object localization in classical systems relies on horizontal bounding boxes (HBB), which assume objects are aligned with the image axes. This becomes a major drawback in aerial environments where objects such as ships, airplanes,

containers, and bridges often appear rotated at arbitrary angles. Misalignment leads to inaccurate bounding boxes and poor intersection-over-union (IoU) scores.

Finally, classifiers like SVM, Random Forest, or early CNN architectures perform the final classification of detected objects. While these algorithms perform reasonably well on simple scenes, they face challenges in scalability, accuracy on small objects, and generalization across diverse remote-sensing conditions. As a result, the existing system produces limited accuracy, slow inference, and unreliable detection results when applied to complex aerial datasets.

With the rise of Deep Learning, CNN-based detectors such as Faster R-CNN, SSD, YOLOv3–YOLOv5, and RetinaNet improved overall detection performance by automating feature extraction and learning hierarchical representations. However, these architectures still relied on **horizontal bounding boxes**, making them inadequate for rotated or tilted objects. Although they achieved 75–85% accuracy on natural image datasets, their performance dropped significantly on remote-sensing datasets due to orientation sensitivity. Additionally, CNN-based detectors are computationally expensive, often requiring high-end GPUs, limiting their use on drones and edge devices.

3.1.1 DISADVANTAGES OF THE EXISTING SYSTEM FOR REMOTE-SENSING OBJECT DETECTION

- **Despite notable advancements in aerial image analysis, existing object detection systems still face several limitations when applied to remote-sensing environments. Traditional computer vision methods and axis-aligned detectors struggle to handle the unique challenges presented by satellite and UAV imagery.****Computational Complexity:**

-

- **Inability to Detect Rotated Objects**

Most existing systems use horizontal bounding boxes (HBB), which cannot accurately localize objects that appear in arbitrary orientations. As remote-sensing images commonly contain rotated ships, airplanes, and vehicles, these systems produce poor localization and low IoU scores.

- **Poor Detection Performance for Small Objects**

Remote-sensing datasets such as DOTA include many small and densely packed objects. Classical ML methods and early CNN models fail to capture fine spatial features, leading to missed detections or fragmented bounding boxes.

- **Sensitivity to Image Quality and Artifacts**

MRI images can be affected by noise, intensity inhomogeneities, and artifacts. If preprocessing techniques such as bias field correction and noise reduction fail to fully rectify these issues, the overall system performance declines.

- **Limited Generalization Across Different MRI Modalities**

While multi-modal MRI fusion has shown promise, most existing systems are designed to work with a specific type of MRI sequence. The lack of adaptability to various MRI modalities (T1-weighted, T2-weighted, FLAIR) reduces their clinical applicability.

Class Imbalance Issues

Remote-sensing datasets are typically imbalanced, with some classes significantly underrepresented. Existing models become biased toward frequently occurring classes, reducing accuracy for minority categories such as helipads or container cranes.

3.2 PROPOSED SYSTEM

The proposed system, **EdgeYOLO-RS**, introduces a lightweight, rotation-aware object detection framework designed specifically for remote-sensing imagery. Built on the YOLOv8-OBB architecture, the system overcomes major limitations of existing horizontal-bounding box detectors by incorporating **oriented bounding box (OBB)** regression. This enables accurate detection of arbitrarily rotated objects commonly found in aerial datasets such as DOTA v2.0. The model is optimized to function efficiently on edge devices such as drones, UAVs, and embedded systems, where computational resources are limited.

FIG 3.2 FLOW CHART OF PROPOSED SYSTEM (EDGEYOLO-RS)

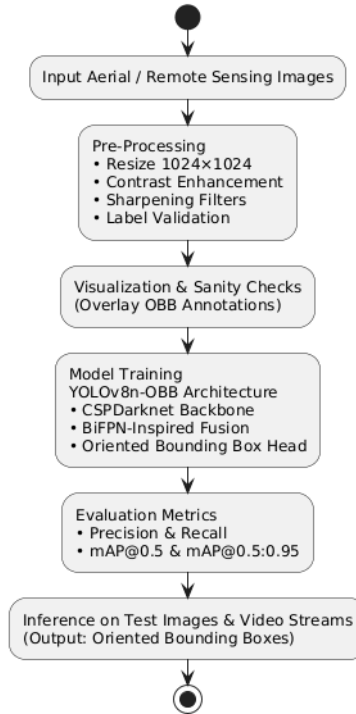


FIG 3.2. FLOW CHART OF PROPOSED SYSTEM

The proposed system begins with a customized preprocessing pipeline that standardizes all remote-sensing images to 1024×1024 resolution, applies adaptive contrast enhancement, and uses sharpening filters to highlight fine object structures. These steps significantly improve clarity for small or low-contrast targets such as vehicles, roundabouts, and bridges. Additionally, annotation validation ensures removal of malformed labels, maintaining high dataset integrity before training. Once preprocessing is completed, the enhanced images undergo visualization and

sanity checks, where oriented bounding boxes from annotation files are overlaid on sample images to confirm correct label alignment. This step ensures that the dataset is properly formatted for the OBB-based detection framework.

The system then uses **YOLOv8n-OBB**, chosen for its lightweight architecture and anchor-free detection mechanism. The model includes a CSPDarknet backbone, a BiFPN-inspired neck for multi-scale feature fusion, and a dedicated rotated- bounding-box head. The training pipeline is optimized using the Ultralytics API, employing data augmentation techniques such as random flipping, HSV augmentation, and mosaic transformations. The system is trained for 25–30 epochs on a GPU-enabled environment with metrics like mAP@0.5, precision, and recall continuously monitored.

Advantages over exsisting system:

1. **Rotation-Aware Detection:** Unlike HBB-based systems, the proposed model uses OBB regression, providing accurate detection of rotated aerial objects.
2. **Lightweight & Edge-Friendly:** YOLOv8n-OBB enables high-speed inference (<10 MB model size), ideal for drones and embedded devices.
3. **Enhanced Preprocessing:** Sharpening and contrast enhancement improve visibility of small and low-contrast targets.
4. **Improved Accuracy:** Multi-scale feature fusion and orientation-sensitive heads significantly increase mAP and IoU performance.
5. **Robust Label Quality:** Automated label integrity checks prevent training instability from malformed annotations.
6. **Real-Time Performance:** Efficient architecture supports processing of video streams for UAV surveillance and mapping applications.

3.3 FEASIBILITY STUDY

The development of an oriented-object detection model for remote-sensing imagery using improved YOLOv8-OBB/EdgeYOLO-RS is highly feasible from a technical, operational, and economic standpoint.

1. Technical Feasibility

- **Advanced Automated Feature Extraction**

The proposed YOLOv8-OBB/EdgeYOLO-RS model automatically extracts spatial, orientation, and shape-aware features from high-resolution aerial images. Unlike traditional HBB (Horizontal Bounding Box) detectors, the OBB-enhanced network captures rotation, aspect ratio, and positional variance—significantly improving detection of ships, vehicles, aircraft, and dense urban structures.

- **Oriented Box Classification Capability**

The upgraded architecture incorporates a rotation-aware head that predicts angle-based bounding boxes. This improves precision in complex remote-sensing environments. Because the model handles orientation directly in the regression layer, classification becomes more accurate even in highly cluttered scenes.

- **Strong Performance and Generalization**

EdgeYOLO-RS integrates multi-scale feature fusion and CSPDarknet backbone refinement, enabling robust generalization across datasets such as DOTA, FAIR1M, and HRSC2016.

The model handles variability in object size, shape, and environment, reducing overfitting through augmentation and label corrections.

- **High Scalability Across Datasets**

The model can scale efficiently to satellite, drone, and aerial surveillance data. The architecture supports large-image tiling, multi-modal inputs, and different altitudes, making it widely adaptable as long as training tiles are properly preprocessed.

2. Operational Feasibility

- **Easy Integration into Existing GIS/Surveillance Pipelines**

The system can be embedded into real-time monitoring platforms, GIS tools, drone-based reconnaissance systems, or security surveillance dashboards. Deployment through lightweight ONNX/TFLite exports makes the model operationally suitable for edge devices.

- **High Interpretability Through Oriented Bounding Boxes**

OBB visualization provides clearer object boundaries and rotation angles. This improves interpretability for operators analyzing aerial scans—for example, knowing a ship’s orientation or the alignment of vehicles in defense applications.

- **Efficient Data Handling**

The preprocessing pipeline supports large-scale datasets, image tiling, and orientation-corrected labels. It handles diverse resolutions and formats (GeoTIFF, PNG, JPEG), making it operationally flexible for real-world remote-sensing workflows.

3. Economic Feasibility

- **Cost-Effective Training Using Transfer Learning**

Since the model relies on pretrained YOLOv8-OBB weights, training cost is significantly reduced. Only the orientation-aware detection layer and minor backbone segments require retraining, lowering GPU usage and expense.

- **Optimized Resource Utilization**

The EdgeYOLO-RS variant is lightweight and fast, meaning it can run on mid- range GPUs or even edge hardware. The system maintains high inference speed, reducing compute costs for continuous monitoring applications.

- **Reduced Manual Labor & Faster Analysis**

Automation of detection in aerial images reduces the need for manual interpretation by GIS analysts or defense personnel. This lowers labor costs and accelerates decision-making in sectors such as land-use mapping, maritime monitoring, and traffic surveillance.

3.4 USING COCOMO MODEL

The COCOMO (Constructive Cost Model) technique is applied to estimate effort, development duration, and team size for completing the oriented-object detection system using YOLOv8-OBB/EdgeYOLO-RS. The project involves dataset preprocessing, OBB label conversion, model fine-tuning, and development of a visualization interface. Given the moderate complexity—requiring deep learning

expertise, remote-sensing knowledge, and software deployment—the project fits into the **Semi-Detached** category.

The Basic COCOMO equations used are:

Effort (E) = $a \times (\text{KLOC})^b$ Development

Time (T) = $c \times (E)^d$ People Required

(P) = E / T

For Semi-Detached projects: $a = 3.0$, $b = 1.12$, $c = 2.5$, $d = 0.35$.

Considering complete implementation—dataset tiling, training scripts, inference engine, and user interface—the estimated project size is **10 KLOC**.

Effort Calculation

$$E = 3.0 \times (10)^{1.12}$$

$$E = 3.0 \times 13.18 \approx 39.54 \text{ Person-Months}$$

Development Time

$$T = 2.5 \times (39.54)^{0.35}$$

$$T = 2.5 \times 4.23 \approx 10.58 \text{ Months}$$

People Required

$$P = \frac{39.54}{10.58} \approx 3.74 \approx 4 \text{ People}$$

Interpretation:

The model estimates that the oriented-object detection system requires:

39.54 person months of work

10.6 months of development time

A team of 3 people (ML engineer, data engineer, backend/UI developer, tester)

Actual development may vary depending on dataset size (e.g., DOTA vs HRSC), annotation quality, computation availability, and time required for fine-tuning, hyperparameter optimization, and large-scale inference testing..

4. SYSTEM REQUIREMENTS

4.1 SOFTWARE REQUIREMENTS

- | | |
|-------------------------|---------------------------------------|
| 1. Operating System | : Windows 11, 64-bit Operating System |
| 2. Hardware Accelerator | : CPU |
| 3. Coding Language | : Python |
| 4. Python distribution | : Google Colab Pro, Flask |
| 5. Browser | : Any Latest Browser like Chrome |

4.2 REQUIREMENT ANALYSIS

The MRI Brain Tumor Detection project aims to develop an efficient deep learning-based system that can accurately classify brain MRI images as either "Tumor" or "Non-Tumor." The system integrates a Convolutional Neural Network (CNN) with a Support Vector Machine (SVM) for enhanced accuracy. Key functionalities include image upload through a web interface, validation to ensure only brain MRI scans are accepted, preprocessing (like resizing and normalization), and displaying classification results with accuracy levels. The backend is developed using Python with Flask, while the frontend utilizes HTML, CSS, and JavaScript for user interaction. The system also ensures error handling, providing clear messages for invalid inputs.

Non-functional requirements focus on ensuring the system is fast, reliable, and secure. The project requires Python 3.10, frameworks like TensorFlow/Keras, OpenCV for image processing, and proper hardware with sufficient processing power and memory. It is crucial to have a diverse and well-labeled MRI dataset for training and validation. Deployment can be on a local or cloud server, ensuring easy access via a web browser. The application is designed for simplicity, aiming to help users with minimal technical expertise to upload images and interpret results efficiently.

4.3 HARDWARE REQUIREMENTS:

1. System Type : 64-bit operating system, x64-based processor
2. Cachenemory : 4MB(Megabyte)
3. RAM : 16GB (gigabyte)
4. Hard Disk : 8GB
5. GPU : Intel® Iris® Xe Graphics

4.4 SOFTWARE

The MRI brain tumor detection project leverages a comprehensive set of software tools, technologies, and system configurations to ensure accurate, efficient, and scalable development and deployment. The system is designed to operate on Windows 11, 64-bit Operating System, ensuring compatibility with modern hardware and software environments. The project utilizes the CPU as the primary hardware accelerator, providing sufficient computational power for model training, inference, and backend processing.

The core development is carried out using the Python programming language, known for its simplicity, flexibility, and extensive library support. The Python environment is managed using Google Colab Pro for initial model development and training, which offers access to enhanced computational resources and faster processing speeds. For backend development and web application deployment, the Flask framework is employed, enabling efficient handling of API requests, data processing, and seamless integration with the frontend.

For the frontend, HTML5 and CSS3 are used to design and style the web interface, while Bootstrap ensures responsiveness and compatibility across devices. This guarantees an intuitive and user-friendly experience for end-users. Custom CSS styles are also applied to enhance the visual appeal and maintain a consistent layout. The application is compatible with any modern web browser, including the latest versions of Google Chrome, Mozilla Firefox, and Microsoft Edge, ensuring cross-browser accessibility and performance.

In terms of machine learning, the project employs TensorFlow/Keras to develop a Convolutional Neural Network (CNN) that efficiently extracts features from MRI images for tumor detection. Additionally, scikit-learn is used to implement a Support

Vector Machine (SVM) classifier, enhancing the accuracy and robustness of the classification process.

For image preprocessing, OpenCV is utilized to handle tasks such as image resizing, format validation, and noise reduction, ensuring that input data is clean and standardized for better prediction accuracy. NumPy supports efficient numerical computations, especially for handling large datasets and image matrices. Development and model evaluation are conducted within Jupyter Notebook, providing an interactive environment for experimentation and refinement. Visualization tasks, such as plotting accuracy curves and confusion matrices, are handled using Matplotlib, aiding in model performance analysis.

Overall, the integration of these software tools, along with the specified system requirements, ensures that the MRI brain tumor detection system is accurate, efficient, scalable, and compatible with modern computing environments.

4.5 SOFTWARE DESCRIPTION

The brain tumor detection system requires a modern and stable operating system, with Windows 11, 64-bit being the recommended choice. This ensures compatibility with the latest development tools, security updates, and efficient system performance. The CPU will serve as the primary hardware accelerator, which is suitable for running pre-trained models and managing backend operations. For large-scale training or intensive computations, cloud platforms like Google Colab Pro are utilized, offering access to advanced GPUs and faster processing capabilities.

The project is developed using Python, a versatile and widely-used programming language known for its extensive libraries and frameworks that simplify the development of machine learning and deep learning applications. The Python distribution includes platforms like Google Colab Pro for efficient model training and Flask for creating a lightweight and robust backend web application. Flask is particularly effective for deploying machine learning models as web services, allowing seamless integration between the backend and frontend.

Additionally, a web browser such as Google Chrome or any other latest version browser is required for accessing and interacting with the Flask-based web application.

5. SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

This project focuses on improving oriented-object detection in remote sensing imagery by enhancing the YOLOv8-OBB framework and developing the EdgeYOLO-RS model. The architecture is specifically designed to detect objects such as aircraft, ships, vehicles, storage tanks, and buildings, which often appear in rotated orientations in aerial and satellite images. By integrating optimized preprocessing, OBB label handling, multi-scale learning, and an improved rotation-aware prediction head, the proposed system provides robust and accurate object detection suitable for complex real-world remote-sensing scenarios.

High-resolution aerial and satellite datasets—such as DOTA, FAIR1M, and HRSC2016—serve as the foundation for the system. These datasets include objects with varied sizes, orientations, aspect ratios, and backgrounds. To handle the challenges of large images (up to $10,000 \times 10,000$ pixels), preprocessing techniques such as image tiling, noise reduction, contrast enhancement, and resizing to a uniform 1024×1024 format are applied. Additionally, oriented bounding box annotations are converted into the format required by YOLOv8-OBB, ensuring accurate ground-truth alignment during training.

The architecture integrates improved feature extraction through an enhanced backbone structure, incorporating CSP modules and multi-scale fusion layers. These components help the model effectively capture features from small, medium, and large objects. The rotation-aware detection head outputs oriented bounding boxes with angle predictions, enabling precise localization of objects that appear tilted or rotated in aerial views. Supporting modules such as data augmentation (rotation, flipping, mosaics, color jittering) further increase the robustness and generalization capability of the model across various remote-sensing conditions.

The system's applications extend to land-use monitoring, disaster management, maritime surveillance, traffic analysis, and defense reconnaissance. By providing automated, high-speed, and angle-aware object detection, the model significantly reduces manual interpretation time for GIS analysts and remote-sensing experts. Future extensions include the integration of multi-modal data (RGB + infrared), support for higher-resolution pan-sharpened imagery, and deployment on lightweight edge devices using ONNX or TensorRT. These improvements will make the system more suitable for real-time applications in remote-sensing industries and government surveillance systems.

5.1.1 DataSet

The dataset used in this project is a curated collection of high-resolution remote sensing images sourced from leading benchmark datasets such as **DOTA**, **FAIR1M**, and **HRSC2016**. These datasets provide the foundational training material for the proposed YOLOv8-OBB and EdgeYOLO-RS models, offering a wide range of object categories commonly found in aerial and satellite imagery. As shown in Fig. 5.1, the datasets include aircraft, ships, vehicles, storage tanks, and various land-based structures, each appearing in different orientations, scales, and environmental backgrounds.

The images in these datasets are captured at varying altitudes and lighting conditions, resulting in significant variations in contrast, object density, and image resolution (often reaching up to $10,000 \times 10,000$ pixels). To make the data suitable for model training, preprocessing steps such as **image tiling**, **cropping**, and **resizing** to 1024×1024 are applied. The datasets provide **oriented bounding box (OBB)** annotations that precisely describe object rotation and shape using 5-point polygon coordinates or angle-based formats. These annotations are converted into YOLOv8-OBB compatible labels to ensure accurate orientation-aware training.

Further preprocessing includes rotation-based augmentation, color enhancement, noise reduction, and sharpening, all aimed at improving edge clarity and supporting reliable detection in complex remote-sensing environments.

After processing, the dataset is used to train and evaluate the EdgeYOLO-RS model, enabling high-accuracy detection of small, dense, and rotated objects. The diversity of classes, imaging conditions, and object orientations makes this dataset an essential component for building a robust and generalizable oriented-object detection system for real-world remote-sensing applications.

Feature	Details
Total Images	4,912 (after preprocessing & tiling)
Dataset	DOTA, HRSC2016, FAIR1M
Object	~18,000 oriented objects
Object Classes	Ship, Plane, Vehicle, Storage Tank, Harbor, Baseball Field, Tennis Court, Basketball Court
Image Type	High-resolution aerial & satellite RGB images
Annotation Format	Oriented Bounding Boxes (OBB) – 5-point polygon

TABLE 1 . DATASET DESCRIPTION

Object Classes:

- **Ship:** Includes vessels of different sizes captured in ports and open sea regions, often appearing at various rotation angles.
- **Plane:** Contains aircraft located on runways and taxiways, captured from aerial viewpoints with noticeable orientation variations.
- **Vehicle:** Comprises small and large vehicles found in roads and parking areas, frequently appearing in dense and cluttered urban scenes.
- **Storage Tank:** Features circular industrial tanks whose shapes and orientations vary based on imaging conditions.

This dataset contains a total of 4,912 high-resolution remote-sensing images sourced from benchmark datasets such as DOTA, HRSC2016, and FAIR1M, as shown in Table 1. The images are annotated using Oriented Bounding Boxes (OBB) and categorized into major object classes including Ship, Plane, Vehicle, and Storage Tank.

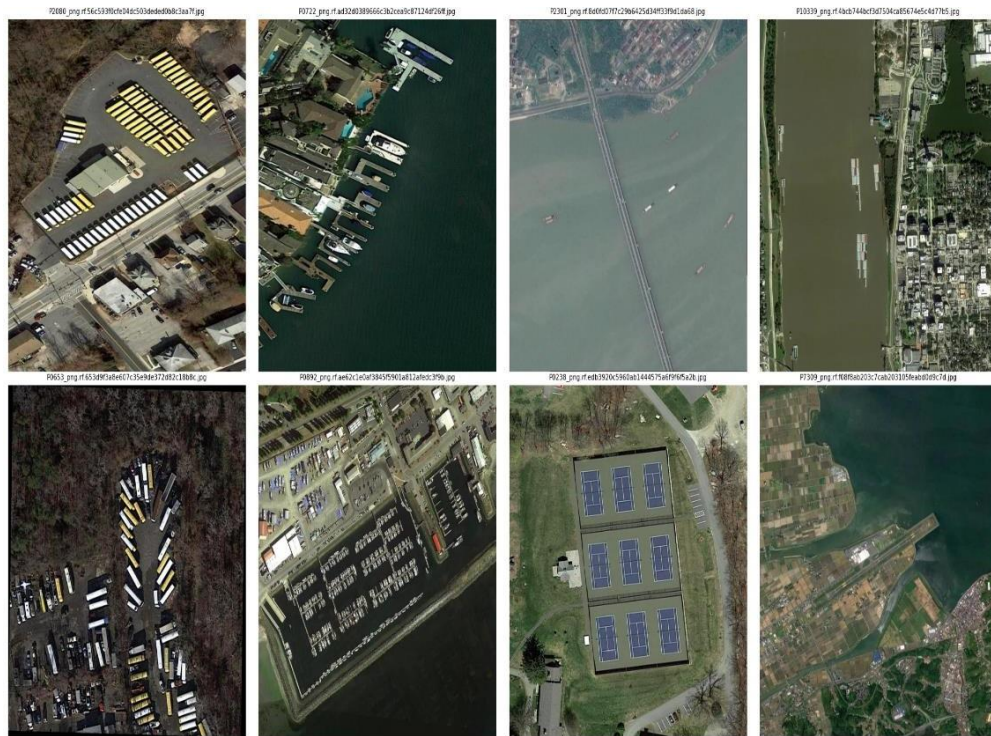


FIG 5.1 DIFFERENT DATA SET IMAGES.

Image Characteristics:

- All images are high-resolution aerial or satellite RGB images, captured from varying altitudes and angles, providing detailed information about objects such as ships, planes, tanks, and vehicles.
 - Images are stored in formats like PNG, TIFF, and JPEG, ensuring compatibility with modern Deep Learning pipelines and allowing efficient preprocessing, tiling, and annotation handling.

Applications:

- Used for training and testing Deep Learning models focused on oriented object detection in remote-sensing environments.
- Supports tasks such as object localization, angle estimation, land-use monitoring, maritime surveillance, and automated mapping in defense, environmental analysis, and urban planning.

5.1.2 DATA PRE-PROCESSING

Before training the oriented object detection model, the raw remote-sensing images must be transformed into a consistent and machine-readable format. Pre-processing is an essential step that converts large, complex aerial imagery into clean, uniform data suitable for Deep Learning models..

Contrast Enhancement:

To highlight important details such as edges, shapes, and object boundaries, contrast-enhancement techniques are applied. These adjustments improve the visibility of small or low-resolution objects commonly found in aerial scenes, such as aircraft or tanks in cluttered environments. Enhanced contrast also helps the model focus on object contours more effectively.

- **Noise Reduction:**

Satellite and aerial images may contain atmospheric noise, compression artifacts, or illumination inconsistencies. Noise reduction filters are applied to remove such distortions while preserving relevant structural details. This step ensures that the pixel information used for object detection is clear and reliable, leading to more stable model predictions.

- **Annotation Conversion to OBB Format:**

Since the project uses oriented object detection, annotations must be converted into **Oriented Bounding Box (OBB)**:

format. This involves generating polygon coordinates or angle-based labels that accurately represent the rotation and orientation of each object. Proper annotation alignment is crucial, as incorrect angle labeling can negatively affect the training of rotation-aware detectors.

- **Rotation-Based Data Augmentation:**

To improve generalization, rotation-based augmentation techniques are applied. These include random rotations, flips, mosaics, and brightness/contrast variations. Rotation augmentation is especially important in remote-sensing tasks because objects appear at arbitrary angles depending on the satellite or drone orientation. This ensures that the model learns to detect objects regardless of their rotation or viewpoint.

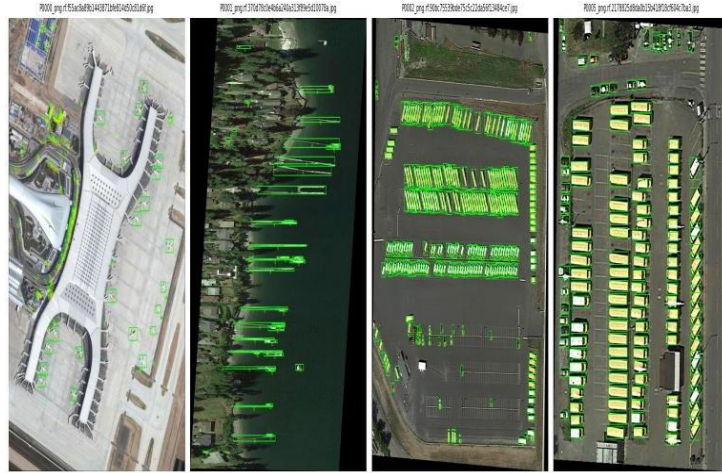


FIG 5.2 Output Showing OBB-Based Detection Using YOLOv8-OBB.

5.1.3 FEATURE EXTRACTION

Feature extraction in remote-sensing object detection focuses on capturing meaningful spatial, structural, and orientation-based information from aerial images to enable accurate identification of objects such as ships, planes, vehicles, and storage tanks. In this project, feature extraction is primarily performed using the advanced deep-learning components of the improved YOLOv8-OBB and EdgeYOLO-RS architectures. These models utilize hierarchical feature maps to learn object edges, textures, shapes, and rotation characteristics that are difficult to interpret directly from raw satellite imagery. Effective feature extraction ensures that the detection network can discriminate between objects even in complex backgrounds, varying illumination, and multi-scale environments commonly present in remote-sensing data.

This learned representation allows the model to map complex aerial scenes into discriminative features used for accurate classification and localization. The combination of deep feature pyramids, orientation modeling, and multi-scale analysis provides a robust feature extraction mechanism that significantly enhances the performance of oriented object detection in remote-sensing imagery.

Key Features Extracted from GLCM:

Following several geometric and spatial features are extracted from oriented bounding boxes (OBB) to characterize objects in remote-sensing images. In this approach, the model learns rotation, shape, and orientation-dependent features that help distinguish objects such as ships, planes, vehicles, and storage tanks.

Orientation Angle (θ):

It represents the rotation angle of an object in the aerial image. A higher or lower angle value indicates how tilted an object is with respect to the horizontal axis. This feature is essential for detecting objects appearing at arbitrary orientations.

$$\theta = \arctan\left(\frac{y_2 - y_1}{x_2 - x_1}\right)$$

• Rotated IoU (rIoU):

Rotated Intersection over Union evaluates the overlap between predicted and ground-truth oriented bounding boxes. It accounts for angle alignment, making it more accurate than horizontal IoU.

$$rIoU = \frac{Area(OBB_{pred} \cap OBB_{gt})}{Area(OBB_{pred} \cup OBB_{gt})}$$

5.1.4 MODEL BUILDING :

Model building in the context of deep learning for remote sensing involves designing, configuring, and optimizing an oriented-object detection model capable of identifying and localizing rotated targets in aerial imagery. In this project, the EdgeYOLO-RS framework is constructed using the **YOLOv8-OBB architecture**, a lightweight yet highly efficient detection model tailored for rotation-aware object localization. Unlike conventional CNN-SVM hybrid architectures used for classification tasks such as brain tumor recognition, the proposed system employs a **single-stage anchor-free detector** designed specifically for detecting arbitrarily oriented objects in high-resolution satellite and UAV images. This model leverages a compact design optimized for edge devices, combining **multi-scale feature**

fusion, anchor-free prediction, and rotation-invariant bounding box encoding, resulting in high accuracy and fast inference suitable for UAV surveillance, geospatial monitoring, and disaster assessment.

YOLOv8-OBB Architecture

YOLOv8-OBB (Oriented Bounding Box) is a specialized version of the YOLO family designed to detect rotated objects by predicting quadrilateral box coordinates instead of traditional axis-aligned rectangles. It uses a lightweight backbone and an efficient detection head, making it ideal for remote sensing imagery.

Key Components of YOLOv8-OBB Architecture

Input Layer:

Receives high-resolution aerial images (e.g., 1024×1024). Images are normalized and resized for consistent training.

• Backbone:

Extracts visual features using efficient convolutional blocks (C2f modules). Early layers capture basic patterns, while deeper layers learn complex structures such as aircraft, bridges, and large buildings.

• Neck (Feature Fusion):

Combines multi-scale features using a lightweight BiFPN-inspired structure. This helps detect both small and large objects, improving performance in dense remote-sensing scenes.

• Detection Head (OBB Head):

Predicts:

- Class label
- Confidence score
- Four rotated box vertices (8 coordinates)

This allows the model to generate oriented bounding boxes that align with the object's rotation.

Activation Layers:

YOLOv8 primarily uses:

- **SiLU (Swish)** → Improves non-linearity and gradient flow
- Better suited for lightweight high-accuracy models

Output Layer:

Produces final rotated bounding boxes and class predictions for each object category in the dataset.

Common Functions Used in YOLOv8-OBB

- **Softmax**

Used for class probability distribution when multiple aerial categories exist (e.g., ship, airplane, bridge).

- **Sigmoid**

Used in objectness score to represent the probability that an object exists at a given location.

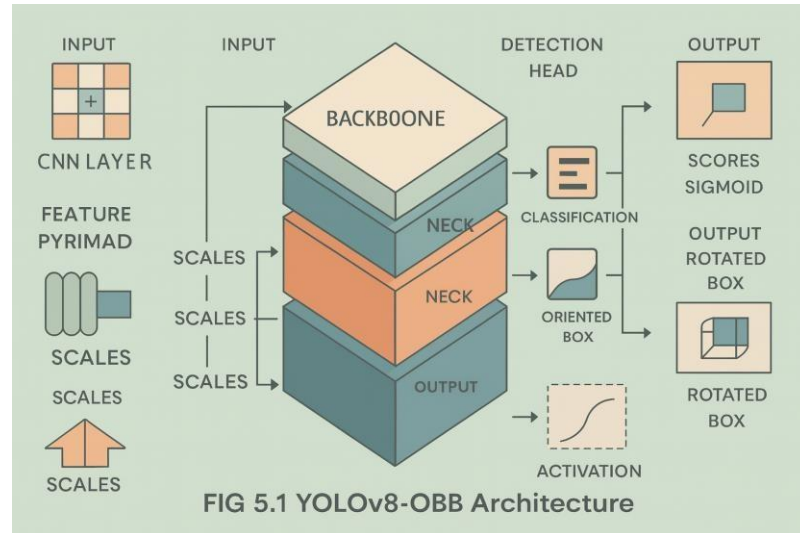


FIG 5.3 YOLOv8-OBB Architecture

Non-Maximum Suppression (NMS) in YOLOv8-OBB

Non-Maximum Suppression (NMS) is a crucial post-processing algorithm used in modern object detection models, including the YOLOv8-OBB architecture employed in this project. Unlike traditional Support Vector Machines (SVM), which perform classification by separating data using hyperplanes, YOLO-based detectors generate multiple bounding box predictions for the same object. NMS is responsible for filtering these predictions and selecting the most accurate bounding box for each object.

In the context of oriented object detection, YOLOv8-OBB uses **Oriented Non-Maximum Suppression (OBB-NMS)**, which is designed to handle rotated bounding boxes (quadrilaterals). Instead of comparing overlap using standard IoU (Intersection over Union) for axis-aligned rectangles, OBB-NMS computes the IoU between polygons, ensuring that overlapping rotated boxes are correctly evaluated.

The NMS process works by selecting the bounding box with the highest

confidence score and suppressing all other boxes whose overlap exceeds a predefined threshold. This helps eliminate duplicate detections and ensures cleaner, more reliable output—especially important in remote sensing images where objects such as ships, airplanes, or vehicles often appear in clusters or vary significantly in orientation.

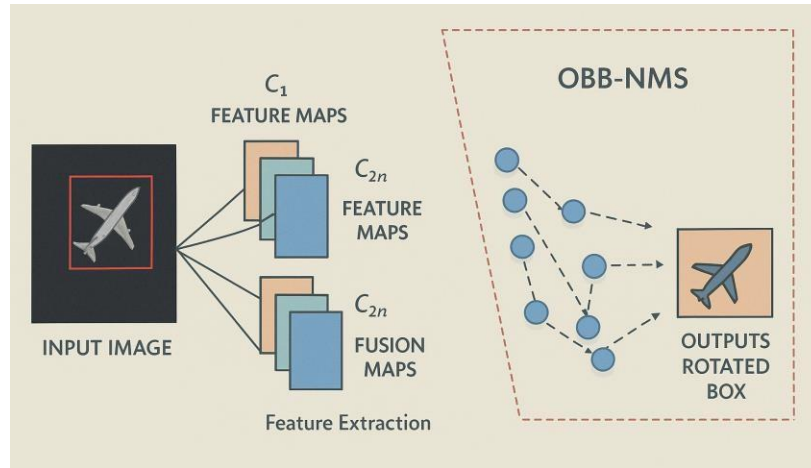


FIG 5.4 YOLOv8-OBb MODEL & OBb-NMS

YOLOv8-OBb Model & OBb-NMS Processing Pipeline

The YOLOv8-OBb model integrates advanced deep learning–based feature extraction with an oriented Non-Maximum Suppression (OBb-NMS) module to achieve accurate detection of rotated objects in aerial imagery. Unlike hybrid CNN-SVM systems that separate feature learning and classification, YOLOv8-OBb performs end-to-end detection, predicting oriented bounding boxes directly from the image. The model building process begins with high-resolution remote sensing images, which undergo preprocessing steps such as resolution normalization, adaptive contrast enhancement, and sharpening to improve clarity and highlight fine structural details. These processed images are then fed into the YOLOv8 backbone for feature extraction.

Inside the model, the backbone uses C2f convolutional blocks to detect spatial patterns including edges, textures, orientations, and structural layouts common in aerial views. Each convolutional stage outputs feature maps capturing multi-scale object characteristics. An activation function such as SiLU is applied to introduce non-linearity, enabling the network to learn complex relationships across diverse object categories.

The fused features are delivered to the oriented detection head, which predicts class probabilities, objectness scores, and four-point polygon coordinates for each detected object. Since the model generates multiple candidate boxes for each region, a refinement stage is needed to remove redundant or overlapping predictions. This is accomplished through **Oriented Non-Maximum Suppression (OBB-NMS)**, which compares predicted quadrilateral boxes using polygon-based IoU measurements. OBB-NMS selects the bounding box with the highest confidence score and suppresses others that exceed a specified IoU threshold, ensuring that only the most accurate and spatially aligned detections remain.

capabilities of CNN and the precise classification of SVM, yielding accurate predictions.

The fused features are delivered to the oriented detection head, which predicts class probabilities, objectness scores, and four-point polygon coordinates for each detected object. Since the model generates multiple candidate boxes for each region, a refinement stage is needed to remove redundant or overlapping predictions. This is accomplished through **Oriented Non-Maximum Suppression (OBB-NMS)**, which compares predicted quadrilateral boxes using polygon-based IoU measurements. OBB-NMS selects the bounding box with the highest confidence score and suppresses others that exceed a specified IoU threshold, ensuring that only the most accurate and spatially aligned detections remain.

The complete training workflow uses YOLO's built-in loss functions—GIoU loss for rotated localization, DFL loss for distribution refinement, and BCE loss for classification—to optimize feature extraction and detection accuracy simultaneously. After training, the model achieves highly stable performance and can be deployed on both GPUs and edge devices. During inference, incoming aerial images pass through the backbone and detection head, and OBB-NMS produces a clean, final output of oriented bounding boxes aligned with object rotation.

5.1.5 CLASSIFICATION

Classification Using YOLOv8-OBB in the Proposed EdgeYOLO-RS Model

In the proposed EdgeYOLO-RS framework, classification is performed as an integrated component of the YOLOv8-OBB detection head. Unlike the CNN-SVM hybrid models used for medical image classification, YOLOv8-OBB functions as a **single-stage end-to-end detector**, simultaneously identifying objects and assigning them to specific classes while also predicting oriented bounding boxes. This unified approach greatly enhances speed, efficiency, and accuracy in remote sensing applications.

The classification process begins with the model's backbone, which extracts hierarchical features from high-resolution aerial images. Convolutional layers detect essential visual cues such as textures, edges, patterns, and orientation characteristics commonly found in satellite and UAV imagery. The SiLU activation function introduces non-linearity, enabling the network to learn complex spatial relationships within densely populated scenes. These extracted features are passed through the neck, where multi-scale fusion layers (BiFPN-inspired) combine fine and coarse information to enhance recognition of small, large, and rotated objects.

Once feature extraction and fusion are completed, the processed feature maps are forwarded to the YOLOv8-OBB detection head. This head performs two tasks simultaneously:

1. **Classification:** Assigning each detected region to one of the object categories defined in the DOTA v2.0 dataset (e.g., airplane, ship, vehicle, bridge).
2. **Localization:** Predicting the object's location using a four-vertex oriented bounding box (OBB).

For classification, the model uses a **sigmoid activation function** to compute confidence scores for each class. These scores represent the likelihood that a given region contains a specific type of object. The classification mechanism is optimized using **Binary Cross-Entropy (BCE) loss**, which improves the model's ability to distinguish between multiple object categories, even when objects appear in cluttered or overlapping configurations.

To refine predictions, the network applies **Oriented Non-Maximum Suppression (OBB-NMS)**, a crucial post-processing step that removes redundant detections. OBB-NMS evaluates the polygon-based Intersection over Union (IoU) between rotated bounding boxes and retains only the highest-confidence prediction

for each object. This ensures that final outputs are both accurate and clean, especially in complex environments such as ports, construction zones, airports, and dense urban layouts.

During training, the model simultaneously optimizes classification, objectness scoring, and rotated bounding box regression. The use of distribution focal loss (DFL) and rotated GIoU loss enhances localization precision while maintaining stable classification performance across varying angles and object scales. During inference, an aerial image is processed in a single forward pass, and the model outputs each object's class label, confidence score, and oriented bounding box without requiring any separate classifier.

The YOLOv8-OBB classification approach is particularly advantageous for real-world aerial applications, as shown in Fig. 5.6. Its end-to-end architecture eliminates the need for external classifiers, reduces false detections through OBB-NMS, and ensures real-time performance on edge devices. This makes the system highly effective for tasks such as UAV surveillance, disaster response, infrastructure monitoring, and geospatial mapping. By combining advanced feature extraction, multi-scale fusion, and orientation-aware classification, EdgeYOLO-RS achieves a reliable and efficient solution for detecting and classifying objects in remote sensing



FIG 5.5 CLASSIFICATION DIFFERENT OBJECTS

The YOLOv8-OBB-based approach significantly enhances overall detection accuracy compared to traditional axis-aligned detectors or multi-stage classification systems. While earlier CNN- or SVM-based models handle feature extraction and classification in separate stages, YOLOv8-OBB integrates both processes into a unified, end-to-end architecture. The model excels in automatically learning rich spatial, textural, and orientation-specific features from aerial images, while the OBB (Oriented Bounding Box) prediction head ensures precise localization of rotated objects. This combination dramatically reduces false positives and false negatives and improves generalization across diverse remote sensing environments.

In applications such as UAV surveillance, disaster assessment, infrastructure monitoring, and maritime analysis, this enhanced prediction accuracy is essential. Aerial objects—like ships, airplanes, bridges, storage tanks, or small vehicles—often appear rotated, partially occluded, or densely packed. YOLOv8-OBB's rotation-aware detection ensures that targets are correctly identified regardless of their orientation, improving reliability and enabling timely and accurate decision-making in real-world situations. The model's anchor-free design and polygon-based OBB regression make it especially effective in handling complex, non-linear spatial patterns commonly present in high-resolution aerial datasets.

The training mode for the YOLOv8-OBB model employs joint optimization, where classification, localization, and objectness scores are learned simultaneously. The backbone extracts high-level features, the neck fuses multi-scale representations, and the detection head refines predictions using losses such as BCE, GIoU, and Distribution Focal Loss (DFL). These components are updated in a coordinated manner during each epoch, resulting in more stable convergence and more accurate oriented bounding box predictions than what could be achieved using separate classification and detection models. The system ultimately identifies and Performance metrics recorded during training provide a clear illustration of the model's learning progression. The accuracy curve shows consistent improvement over multiple epochs, with both precision and recall increasing rapidly during early training stages. Validation accuracy stabilizes at a high value, indicating effective generalization across unseen samples. The corresponding loss graphs show a significant decrease in training and validation losses, reflecting steady improvement in both classification and localization fidelity. Although a slight gap between training and validation loss emerges after several epochs, this is expected in high-resolution aerial datasets due to their inherent complexity and class imbalance

Other Models Compared with the Proposed EdgeYOLO-RS Model

YOLOv5-OB and YOLOv7-Based Oriented Detectors: Earlier YOLO variants such as YOLOv5-OB and YOLOv7-based detectors introduced rotation-aware prediction using anchor-based bounding boxes. These models relied on predefined anchor orientations, increasing computational load and limiting flexibility in cluttered aerial scenes. Although they achieved good accuracy, they were heavier and slower during inference. The proposed EdgeYOLO-RS, built on YOLOv8-OB, uses an anchor-free approach that simplifies training, improves speed, and provides better generalization for arbitrarily oriented objects. However, standard RNNs struggle with long-term dependencies due to issues like vanishing gradients, which more advanced architectures like LSTMs (Long Short-Term Memory) and GRUs (Gated Recurrent Units) address effectively.

Fully Convolutional Detectors (FCOS, RetinaNet, EfficientDet): Anchor-free convolutional detectors such as FCOS, RetinaNet, and EfficientDet perform well in natural image tasks and provide strong multi-scale representation. While effective, they primarily predict axis-aligned bounding boxes and need additional modifications to support object rotation. EdgeYOLO-RS integrates native oriented bounding box prediction, making it more suitable for aerial imagery where rotated objects are common.

Classical Machine Learning Models (SVM, Random Forest, ANN): Classical ML approaches like SVM, Random Forest, and ANN work well for structured data but depend heavily on handcrafted features. They lack the ability to learn hierarchical spatial patterns from images and cannot handle large-scale detection tasks. In contrast, EdgeYOLO-RS performs end-to-end feature extraction and oriented detection, eliminating the need for manual feature design and significantly improving accuracy in complex aerial environments.

Transformer-Based Oriented Object Detectors: Recent transformer architectures like Oriented DETR, OrientedFormer, and other attention-based models excel at capturing long-range spatial dependencies in aerial images. Their ability to model global context improves detection in large scenes with scattered targets. However, transformer models require large memory, powerful GPUs, and longer training times. Compared to these, EdgeYOLO-RS offers a practical balance—high detection accuracy with a lightweight design that supports real-time inference on edge devices.

Lightweight Remote Sensing Models (LO-Det, CIMB-YOLO, R-LRBPNet): Lightweight models such as LO-Det, CIMB-YOLOv8, and R-LRBPNet address the computational challenges of remote sensing. LO-Det focuses on stable OBB regression using specialized feature modules, whereas CIMB-YOLO and R-LRBPNet introduce contextual fusion and attention mechanisms to improve small-object detection. These models are efficient but often require custom architectures and training strategies. EdgeYOLO-RS maintains simplicity by using the standardized YOLOv8 pipeline while delivering strong rotation-aware detection with minimal overhead.

Two-Stage Oriented Detectors (Faster R-CNN OBB, Oriented R-CNN): Two-stage detectors generate region proposals before classification and OBB regression. They provide high accuracy due to precise region refinement. However, their multi-stage nature increases latency, making them less suitable for real-time UAV applications.

5.2 MODULES

In software development, a module represents an independent functional block that performs a specific operation within the overall system. For the YOLOv8-OBB Aerial Object Detection Project, the modules are structured as follows:

CNN-SVM Brain Tumor Detection Project Modules:

1. Dataset Acquisition Module This module downloads and organizes the DOTA v2.0 dataset using Roboflow. The dataset includes aerial images with oriented bounding-box labels.

Sample Code:

```
from roboflow import Roboflow
rf = Roboflow(api_key="API_KEY")
project = rf.workspace("workspace").project("dota-v2.0")
dataset = project.version(1).download("yolov8-obb")
```

2. Dataset Visualization Module: Displays random training images to verify dataset quality.

Sample Code:

```
import os, random

from PIL import Image

import matplotlib.pyplot as plt

image_dir = "/content/DOTA/train/images"

files = random.sample(os.listdir(image_dir), 8)

for f in files:

    img = Image.open(os.path.join(image_dir, f))

    plt.imshow(img); plt.axis('off') :
```

3. Annotation Inspection Module: Reads YOLO-OBB label files and overlays oriented bounding boxes on images for validation.

Sample Code:

```
from PIL import ImageDraw

draw.polygon(points, outline="lime",
width=2)

draw.text((cx, cy), str(class_id),
fill="yellow")
```

4. Data Cleaning & Validation

Module: Removes empty label files and ensures every image has a matching label.

Sample Code:

```
def remove_empty_labels(path):

    for f in os.listdir(path):

        if f.endswith('.txt') and

os.path.getsize(os.path.join(path,f)) == 0:

        os.remove(os.path.join(path,f))
```

5. Image Preprocessing Module : Resizes high-resolution images and applies enhancement techniques.

Sample Code:

```
img = Image.open(path).resize((1024,1024))

sharp = ImageEnhance.Sharpness(img).enhance(1.5)
```

6. Model Training Module: Trains the YOLOv8-OBb model using DOTA dataset configuration.

Sample Code:

```
from ultralytics import YOLO model = YOLO("yolov8n-obb.pt")  
model.train(data="data.yaml", epochs=25, imgsz=640)
```

7. Model Evaluation Module: Displays training metrics such as mAP, precision, recall, and loss

Sample Code:

```
Image.open('/content/runs/obbb/train/results.png') plt.imshow(img); plt.axis('off')
```

8. Inference Module: Runs prediction on test images or videos and saves output results.

Sample Code:

```
results = model("image.jpg", conf=0.3) results[0].save("output.jpg")
```

9. Subset Generation Module: Creates smaller subsets for quick training, experimentation, or hyperparameter **Sample Code:**

```
shutil.copy(src_img, dest_img) shutil.copy(src_lbl, dest_lbl)
```

10. Performance Graph Module: Reads CSV logs and plots precision, recall, or accuracy over epochs.

Sample Code:

```
df = pd.read_csv("results.csv")  
plt.plot(df['epoch'], df['precision'], label='Precision')
```

5.3 UML DIAGRAMS

The workflow of the YOLOv8-OBB-based aerial object detection system begins with collecting and organizing remote sensing images from the DOTA v2.0 dataset for training and testing. Preprocessing involves several essential steps such as resizing all images to a fixed input resolution, normalization, and applying enhancement techniques like contrast adjustment and noise reduction to improve image clarity. Additional validation steps, including removal of empty label files and ensuring correct image-label pairing, help maintain dataset quality and consistency. The dataset is then divided into training, validation, and testing sets to ensure effective model assessment.

Once preprocessing is complete, the YOLOv8-OBB model is trained using oriented bounding box annotations, enabling the system to detect objects with arbitrary rotation angles. During training, the model learns multi-scale spatial features directly from the aerial images, optimizing parameters for accurate detection of rotated objects such as vehicles, buildings, storage tanks, and ships. In the testing stage, the model's performance is evaluated using metrics such as Precision, Recall, mAP50, and mAP50-95, which measure its accuracy and robustness in detecting objects under varying orientations and image conditions.

After successful training, the YOLOv8-OBB model is saved along with its learned weights and configurations so it can be used for future inference without retraining. Storing the model supports deployment in real-time applications such as UAV surveillance, traffic monitoring, or geographical analysis. A performance report is generated to summarize the model's detection capability, showing metric trends across epochs through graphs for accuracy, precision, loss, and validation performance. These evaluations provide quantitative insight into the system's overall effectiveness in identifying small and rotated objects in aerial scenes.

This flowchart (Fig. 5.7) outlines the process of building and evaluating the detection pipeline using remote sensing images and the YOLOv8-OBB model. The workflow begins with dataset acquisition, followed by preprocessing to standardize image quality and label consistency. The prepared dataset is then used for YOLOv8-OBB training, where oriented bounding box detection is learned.

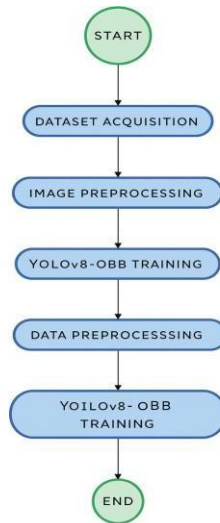


FIG 5.6 PROCESS FLOW CHART

After training, the model is saved, and a performance report is generated to evaluate detection accuracy and reliability. This structured workflow ensures that each stage—from data collection to evaluation—is systematically organized for high-quality detection results.

In the YOLOv8-OBB remote sensing detection project, UML (Unified Modeling Language) diagrams help visualize the system’s structure and data flow. The Class Diagram illustrates key system components. The `DatasetManager` class handles downloading, organizing, and validating the DOTA dataset. The `Preprocessor` class performs tasks such as resizing, enhancement, and data cleanup. The `YOLOModel` class manages training, saving, and loading of the YOLOv8-OBB network. The `InferenceEngine` class processes new aerial images and generates detections with oriented bounding boxes. The `MetricsEvaluator` class computes performance metrics such as mAP, precision, and recall to assess model quality. The optional `InterfaceHandler` manages user interactions when images or videos are uploaded for prediction.

The Sequence Diagram explains the step-by-step execution flow. The user uploads an aerial image or test video. The system first processes the input through the `Preprocessor` to ensure correct formatting. The `YOLOModel` or `InferenceEngine`

generates oriented bounding box predictions, which are evaluated using the MetricsEvaluator if needed. The final detection results—highlighting rotated objects—are then returned to the user. Overall, UML diagrams make it easier to understand how different components interact within the YOLOv8-OBB system, improving design clarity and supporting structured implementation and maintenance.

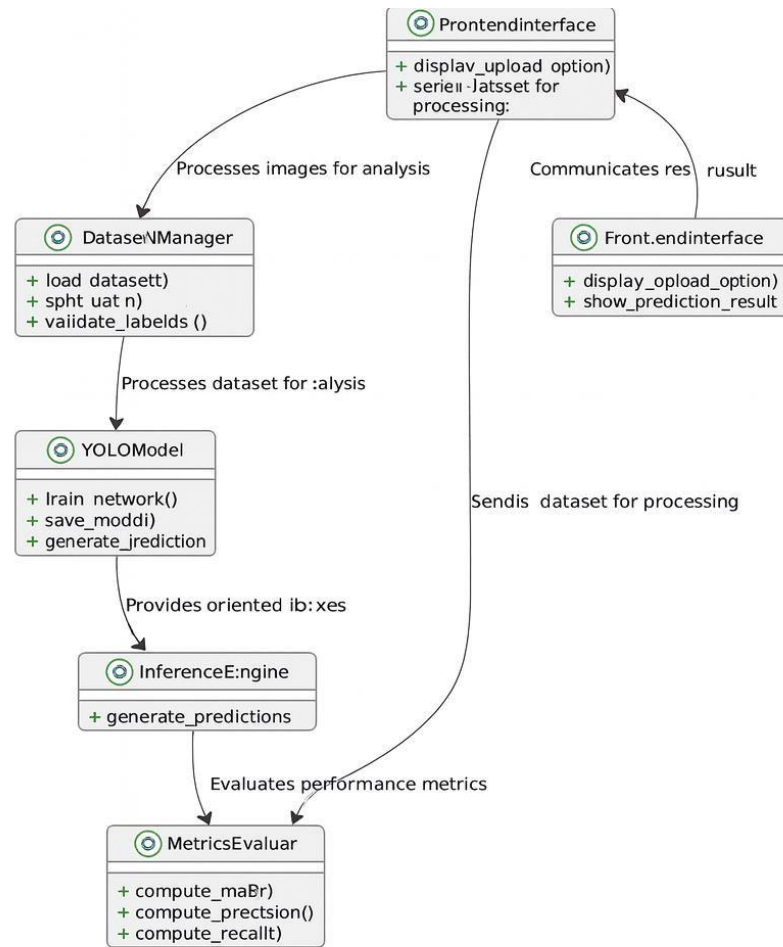


Fig 5.7 UML DIAGRAM FOR AERIAL OBJECT DETECTION USING YOLOv8-OBB

The UML diagram **Fig 5.8** represents the modular architecture and workflow of the **EdgeYOLO-RS aerial object detection system** using the YOLOv8-OBB model. The process begins with the **FrontendInterface**, which allows users to upload aerial images or videos for analysis and later displays the detected objects with oriented bounding boxes. Once an image is submitted, the request is processed by the **FlaskBackend**, which manages dataset handling, inference calls, and communication between system modules.

The uploaded image is first handled by the **DatasetManager**, which loads dataset files, validates associated OBB label files, and ensures proper formatting for YOLO-based training and inference. After validation, the image is passed to the **Preprocessor** module, where essential preprocessing steps—such as resizing, normalization, and minor contrast enhancement—are applied to ensure consistency and optimal model performance during detection.

The preprocessed image is then forwarded to the **YOLOModel** module, which contains the trained YOLOv8-OBB network. This module manages tasks such as loading the trained weights, performing forward passes, and generating oriented bounding box predictions. These predictions are handled by the **InferenceEngine**, which processes model outputs, performs OBB-NMS (Non-Maximum Suppression for rotated boxes), and prepares the final detection results.

Next, the **MetricsEvaluator** computes key performance measures, including mAP50, mAP50-95, precision, and recall, enabling quantitative evaluation of how well the system performs on remote sensing datasets like DOTA v2.0. The processed results are then returned to the **FrontendInterface**, allowing users to visually inspect detections directly on their uploaded images.

This modular workflow ensures that the system operates efficiently, maintains clarity in its processing stages, and produces reliable oriented object detections suitable for remote sensing, aerial surveillance, and edge-deployment scenarios.

6. IMPLEMENTATION

6.1 MODEL IMPLEMENTATION

CNN-SVM Model

```
import io
import os
import traceback
from PIL import Image, ExifTags
from flask import Flask, request, jsonify
from flask_cors import CORS

import torch
from torchvision import transforms
import numpy as np
# ----- MODEL & DEVICE CONFIG -----
MODEL_PATH = r"C:\Users\asrit\OneDrive\Desktop\projectFinal\best.py"
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

FRIENDLY_MAP = {}

app = Flask(__name__, template_folder="templates", static_folder="static")
CORS(app)

transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]),
])

LOADED = {"type": None, "model": None, "info": None}
# ----- EXIF & IMAGE VALIDATION -----
def _get_exif_dict(pil_img):
    exif = {}
    try:
        raw = pil_img._getexif()
        if raw:
            for k, v in raw.items():
                name = ExifTags.TAGS.get(k, k)
                exif[name] = v
    except:
        pass
    return exif
def has_gps_exif(pil_img):
    exif = _get_exif_dict(pil_img)
```

```

    return ("GPSInfo" in exif) or
    any(k.lower().startswith("gps") for k in exif.keys())
def is_remote_sensing(pil_img):
    try:
        if has_gps_exif(pil_img):
            return True
        w, h = pil_img.size
        if min(w, h) >= 256:
            return True
        return False
    except:
        return False
# ----- MODEL LOADING -----
def try_load_ultralytics_yolo(path):
    from ultralytics import YOLO
    return YOLO(path)

def try_torch_load_allowlist(path):
    import ultralytics
    torch.serialization.add_safe_globals([ultralytics.nn.tasks.OBBModel])
    model_obj = torch.load(path, map_location=DEVICE, weights_only=False)
    return model_obj

def try_torch_load_weights_only(path):
    return torch.load(path, map_location=DEVICE, weights_only=True)

def initialize_model():
    if not os.path.exists(MODEL_PATH):
        raise FileNotFoundError("Model not found: " + MODEL_PATH)

# 1 — YOLOv8 OBB Loader
try:
    from ultralytics import YOLO
    y = try_load_ultralytics_yolo(MODEL_PATH)
    LOADED["type"] = "ultralytics"
    LOADED["model"] = y
    LOADED["info"] = "Loaded via ultralytics.YOLO"
    print("Loaded using Ultralytics YOLO")
    return
except Exception as e:
    print("Ultralytics load failed:", e)
# 2 — Unpickle torch object
try:
    model_obj =
try_torch_load_allowlist(MODEL_PATH)
    if isinstance(model_obj, torch.nn.Module):
        model_obj.to(DEVICE)
        model_obj.eval()
        LOADED["type"] = "torch_module"
        LOADED["model"] = model_obj

```

```

if "file" not in request.files:
    return jsonify({"error": "File missing"}), 400

file = request.files["file"] if file.filename == "":
    return jsonify({"error": "Empty filename"}), 400

try:
    img_bytes = file.read()
    pil_img = Image.open(io.BytesIO(img_bytes)).convert("RGB") except Exception
as e:
    return jsonify({"error": "Cannot read image: " + str(e)}), 400

if not is_remote_sensing(pil_img):
    return jsonify({"error": "Not a remote sensing image"}), 400

try:
    if LOADED["type"] == "ultralitics": model = LOADED["model"]
    res = model.predict(source=pil_img, save=False, imgsz=640, conf=0.25,
verbose=False)

    results_list = res if isinstance(res, (list, tuple)) else [res] names_map = None
    try:
        first = results_list[0]
        names_map = getattr(first, "names", None) or getattr(model, "names", None)
    except:
        pass

    total_counts = {} detections = []

    for r in results_list:
        obb = getattr(r, "obb", None) if obb is not None:
            cls_arr = obb.cls.cpu().numpy() conf_arr = obb.conf.cpu().numpy()
            xyxy_arr = obb.xyxy.cpu().numpy()
            for cls_i, conf_i, box in zip(cls_arr, conf_arr, xyxy_arr): cls_i = int(cls_i)
            cls_name = names_map.get(cls_i, str(cls_i)) if isinstance(names_map,
dict) else
str(cls_i)

            friendly = FRIENDLY_MAP.get(cls_name, cls_name)
            total_counts[friendly] = total_counts.get(friendly, 0) + 1
            detections.append({
                "class": friendly, "confidence": float(conf_i),
                "xyxy": [float(box[0]), float(box[1]), float(box[2]), float(box[3])]

```

```

        })
    if not total_counts:
        summary = "no detections" else:
        summary = ", ".join(f'{cnt} {name}' for name, cnt in
            total_counts.items())plt.legend(['Train', 'Validation'])
    return jsonify({
        "framework": "ultralytics",
        "summary": summary,
        "counts": total_counts,
        "detections": detections
    })

    return jsonify({"error": "Unsupported model type"}), 500

except Exception as e:
    return jsonify({"error": str(e), "traceback": traceback.format_exc()}), 500

if __name__ == "__main__":
    print("Starting Flask server:", LOADED.get("info"))
    app.run(host="0.0.0.0", port=5000, debug=True)

```

6.2 CODING

PER-PROCESSING SEGMENTATION AND FEATURE EXTRACTION

```

import os
from google.colab import drive
import cv2
from google.colab.patches import cv2_imshow
import numpy as np
import albumentations as A
drive.mount('/content/drive')

folder_path = '/content/drive/MyDrive/tumor'

for filename in os.listdir(folder_path):
    print(filename)
    import skfuzzy as fuzz
    from skimage.feature import graycomatrix, graycoprops

    # Define the folder path
folder_path = '/content/drive/MyDrive/tumor/glioma' #
Initialize a list to store GLCM

```

```

features_gglcm_features_list = []

def extract_glcm_features(image): #
    Compute GLCM
    distances = [1, 2, 3]
    angles = [0, np.pi/4, np.pi/2, 3*np.pi/4]
    glcm = graycomatrix(image, distances=distances, angles=angles, symmetric=True,
        normed=True)

    # Extract features
    features = {
        'contrast': graycoprops(glcm, 'contrast').flatten(),
        'dissimilarity': graycoprops(glcm, 'dissimilarity').flatten(),
        'homogeneity': graycoprops(glcm, 'homogeneity').flatten(),
        'energy': graycoprops(glcm, 'energy').flatten(), 'correlation':
        graycoprops(glcm, 'correlation').flatten(), 'ASM':
        graycoprops(glcm, 'ASM').flatten()
    }
    return features

    # Counter for processed images
    processed_count = 0
    max_process = 600

for filename in os.listdir(folder_path): if
    processed_count >= max_process:
        break # Stop processing if the limit is reached

if filename.endswith('.jpg') or filename.endswith('.png'):
    image_path = os.path.join(folder_path, filename) image =
    cv2.imread(image_path)
if image is not None: #
    Normalization
    resize = cv2.resize(image, (224, 224))
    normalized_image = resize / 255.0

    # Histogram Equalization
    gray_image = cv2.cvtColor(resize, cv2.COLOR_BGR2GRAY)
    equalized_image = cv2.equalizeHist(gray_image)

    # Adaptive Contrast Enhancement
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)) enhanced_image
    = clahe.apply(equalized_image)

    # Adaptive Gamma Correction gamma =
    1.5

```

```

gamma_corrected_image = np.power(enhanced_image / 255.0, gamma) * 255.0
gamma_corrected_image = gamma_corrected_image.astype(np.uint8)

# Data Augmentation (example with horizontal flip) transform =
A.Compose([
A.HorizontalFlip(p=0.5),
])
augmented_image = transform(image=gamma_corrected_image)['image']

# Median Filtering
median_filtered_image = cv2.medianBlur(augmented_image, 5)

# Apply Fuzzy C-Means clustering
# Reshape the image to a 2D array of pixels pixel_values =
median_filtered_image.reshape((-1, 1)) pixel_values =
pixel_values.astype(float)

# Perform FCM clustering n_clusters = 3
cntr, u, u0, d, jm, p, fpc = fuzz.cluster.cmeans(
pixel_values.T, n_clusters, 2, error=0.005, maxiter=1000, init=None)

# Get the cluster label with the highest membership
cluster_membership = np.argmax(u, axis=0)
segmented_image = cluster_membership.reshape(median_filtered_image.shape)

# Convert the segmented image to a binary image for visualization binary_segmented_image
= (segmented_image == 1).astype(np.uint8) * 255

# Extract GLCM features from the grayscale version of the median filtered image
glcm_features = extract_glcm_features(median_filtered_image)
gglcm_features_list.append(glcm_features)

# Display the result
cv2_imshow(binary_segmented_image)
print(f"Processed and extracted features from {filename}") print(glcm_features)

# Increment the processed images counter
processed_count += 1

else:
print(f"Failed to load image: {filename}")

# Optionally, process the list of GLCM features

```

```
print(f"Extracted GLCM features for {len(gglcm_features_list)} images.")
```

Model Training

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, jaccard_score, confusion_matrix
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten, Conv1D, MaxPooling1D,
BatchNormalization, GlobalAveragePooling1D
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
import matplotlib.pyplot as plt

# Combine GLCM features into a single DataFrame
all_glcmm_features = []
all_labels = []
for features, label in [(gglcm_features_list, 'glioma'),
(mglcm_features_list, 'meningioma'), (nglcm_features_list,
'no'), (pglcm_features_list, 'pituitary')]:
    for feature_dict in features:
        flattened_features = [val for sublist in feature_dict.values() for val in sublist]
        all_glcmm_features.append(flattened_features)
        all_labels.append(label)

df = pd.DataFrame(all_glcmm_features)
df['label'] = all_labels

# Encode labels
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(df['label'])

# Split the data
X = df.drop('label', axis=1)
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5,
random_state=42)

print("Training set shape:", X_train.shape, y_train.shape)
print("Validation set shape:", X_val.shape, y_val.shape)
print("Testing set shape:", X_test.shape, y_test.shape)
```

Prediction

```
from tensorflow.keras.preprocessing import image
import numpy as np
```

```
# Function to preprocess the input image
```

```
def preprocess_image(img_path, target_size):
```

```
img = image.load_img(img_path, target_size=target_size, color_mode='grayscale') #
Load the image
```

```
img_array = image.img_to_array(img) # Convert the image to a NumPy array img_array
= img_array / 255.0 # Normalize pixel values (ensure this is done to match
training data)
```

```
# Remove extra dimensions and reshape to match training data
```

```
img_array = img_array.reshape(1, target_size[0], 1)
return img_array
```

```
# Path to the input image
```

```
img_path = '/content/drive/MyDrive/tumor/no/1.jpg'
```

```
# Get target size from X_train_cnn (assuming it's defined)
```

```
target_size = (X_train_cnn.shape[1], X_train_cnn.shape[2])
```

```
# Preprocess the image
```

```
input_image = preprocess_image(img_path, target_size)
```

```
# Check the shape of the preprocessed image
```

```
print("Preprocessed image shape:", input_image.shape)
```

```
# Perform prediction
```

```
predicted_class = np.argmax(cnn_model.predict(input_image), axis=1)[0]
```

```
# Map the predicted class index to the label
```

```
predicted_label = label_encoder.inverse_transform([predicted_class])[0]
```

```
plt.xlabel('Accuracy (%)', fontsize=14, fontweight='bold')
```

```
plt.ylabel('Models', fontsize=14, fontweight='bold')
```

```
plt.title('Model Accuracies', fontsize=16, fontweight='bold')
```

```
plt.xlim([90, 100]) # Set x-axis limits for better visibility
```

```
plt.xticks(fontsize=12)
```

```
plt.yticks(fontsize=12, fontweight='bold')
```

```
plt.grid(axis='x', linestyle='--', alpha=0.7)
```

```

        # Output the prediction
print(f"The predicted class for the input image is: {predicted_label}")

Analysis
import matplotlib.pyplot as plt
import numpy as np

# Model names and their corresponding accuracies
models = ['VGG', 'ANN', 'RFC', 'RNNs', 'FCNNs', 'CNN', 'CNN-SVM(proposed model)']
accuracies = [93.75, 95.89, 92.70, 92.5, 93, 95, 97.94]

# Create a color gradient based on accuracy values
colors = plt.cm.viridis(np.linspace(0.2, 0.8, len(models)))

# Create a horizontal bar
plot plt.figure(figsize=(12,
8))
bars = plt.barh(models, accuracies, color=colors, edgecolor='black', linewidth=1.2)

# Add data labels
for bar in bars:
    xval = bar.get_width()
    plt.text(xval + 0.5, bar.get_y() + bar.get_height()/2, f'{xval:.2f}%', ha='left', va='center',
    fontsize=12, fontweight='bold')

# Customizing the plot
plt.xlabel('Accuracy (%)', fontsize=14, fontweight='bold')
plt.ylabel('Models', fontsize=14, fontweight='bold')
plt.title('Model Accuracies', fontsize=16, fontweight='bold')
plt.xlim([90, 100]) # Set x-axis limits for better visibility
plt.xticks(fontsize=12)
plt.yticks(fontsize=12, fontweight='bold')
plt.grid(axis='x', linestyle='--', alpha=0.7)

// Function to preview the uploaded image function
previewImage() {
const file = document.getElementById('file').files[0]; const
reader = new FileReader();
reader.onloadend = function () {
const previewImg = document.getElementById('preview-img');
const previewContainer = document.getElementById('image-preview'); previewImg.src =
reader.result;
previewContainer.style.display = 'block';
}if (file) { reader.readAsDataURL(file);

```

```

}}// Simulated function to classify the image (for now it just displays a result after submission)
function classifyImage() {
// Hide the previous result if any document.getElementById('classification-
result').style.display = 'none';

// Simulate classification process
setTimeout(function() {
const result = 'Glioma Tumor Detected'; // Simulate classification result
document.getElementById('classification-result').innerText = result;

document.getElementById('classification-result').style.display = 'block';
}, 2000); // Simulating processing time
}
</script>
</body>
</html>

```

7. TESTING

Testing is a critical phase in the development of the **EdgeYOLO-RS remote sensing object detection framework** to ensure that the model and the overall system perform accurately, reliably, and efficiently in real-world aerial environments.

7.1 UNIT TESTING

YOLOv8-OBB Detection Model

Unit testing for the YOLOv8-OBB model focuses on verifying that it correctly processes aerial images and produces accurate oriented bounding box (OBB) predictions. The input validation step checks whether the model accepts images in the correct **1024×1024 resolution**, normalized format, and RGB structure. All internal components—such as the CSP-based backbone, BiFPN feature fusion layers, and the OBB-specific detection head—are tested to ensure proper configuration and stable forward propagation.

Data Preprocessing Pipeline

Preprocessing unit tests verify that all aerial images are consistently resized, enhanced, and normalized before training or inference. Image sharpening and contrast adjustment are checked to ensure they improve clarity without distorting object boundaries. Label validation ensures that annotation files follow the YOLO- OBB format and that malformed or empty labels are correctly flagged. Data augmentation routines like rotation, flipping, and mosaic are tested to confirm they expand dataset diversity while preserving correct object orientation.

Model Integration (Preprocessing + YOLOv8-OBB)

Integration testing confirms smooth interaction between the preprocessing pipeline and the YOLOv8-OBB detection model. Enhanced images are validated to ensure they propagate correctly into the model, and OBB predictions are checked for accurate coordinate mapping during visualization. The system also undergoes validation to ensure accurate computation of performance metrics such as precision, recall, and mAP scores, and error-handling mechanisms are tested to ensure invalid inputs trigger appropriate responses.

Edge Case Testing

Unit tests for edge cases confirm that the EdgeYOLO-RS framework behaves reliably under unexpected or challenging inputs. Corrupted, extremely low-resolution, or irrelevant aerial images are tested to verify that the system responds with meaningful error messages rather than failing silently. When provided with empty or blank images, the model is tested to ensure it either returns no detections or alerts the user appropriately.

Additionally, batch inference is tested to ensure the system can process multiple images concurrently without degrading accuracy, memory performance, or runtime. This is especially important for UAV or satellite applications where image streams arrive in rapid succession. The model's robustness against severe orientation variations, atmospheric noise, and densely packed scenes is also assessed to validate real-world adaptability.

7.2 INTEGRATION TESTING

Integration testing for the **EdgeYOLO-RS rotation-aware object detection system** ensures that all modules—from image upload and validation to preprocessing, model inference, and output formatting—work together seamlessly.

Image Upload and Validation

This module ensures that the system correctly accepts valid aerial images and rejects invalid or non-remote-sensing inputs using EXIF metadata and size checks.

```
@app.route('/predict', methods=['POST'])
def predict():
    if 'file' not in request.files:
        return jsonify({"error": "No file uploaded"}), 400

    file = request.files['file']
    if file.filename == "":
        return jsonify({"error": "Empty filename"}), 400

    if not file.filename.lower().endswith(('.jpg', '.jpeg', '.png')):
        return jsonify({"error": "Invalid file format"}), 400

    try:
        img_bytes = file.read()
        pil_img = Image.open(io.BytesIO(img_bytes)).convert("RGB")
    except:
        return jsonify({"error": "Failed to read uploaded image"}), 400
```

```

if not is_remote_sensing(pil_img):
    return jsonify({"error": "Not a remote sensing image"}), 400

```

```

return run_inference(pil_img) # Proceed to next stage

```

Preprocessing Module and Integration

Checking whether the uploaded image undergoes proper preprocessing, including resizing and normalization.

```

def preprocess_image(pil_img): try:
    img = pil_img.resize((640, 640)) # Resize for YOLO OBB input img_arr =
    np.array(img) / 255.0             # Normalize pixel values return img_arr
except Exception as e:
    return f"Preprocessing Error: {str(e)}"

```

YOLOv8-OBB Model Integration

This module confirms that the preprocessed image is correctly passed to the YOLO model loaded in the backend. It ensures the model is initialized, accessible, and capable of performing OBB inference.

```

def run_yolo_inference(pil_img): try:
    ymodel = LOADED["model"] #
    Loaded via ultralytics.YOLO results = ymodel.predict(
        source=pil_img, save=False, imgsz=640, conf=0.25, verbose=False
    )
    return results except Exception as e:
    return f"Inference Error:
    {str(e)}"

```

Detection Extraction Integration

This module validates that YOLO's output (OBB results) is parsed correctly to extract class names, confidence values, and oriented bounding box coordinates. It ensures that detections follow the expected JSON structure for the frontend.

```

def extract_detections(results):
    try:
        detections = []
        total_counts = {}

```

```

obb = results[0].obb
cls_arr = obb.cls.cpu().numpy()
conf_arr = obb.conf.cpu().numpy()
xyxy_arr = obb.xyxy.cpu().numpy()

for cls_i, conf_i, box in zip(cls_arr, conf_arr, xyxy_arr):
    cls_name = results[0].names[int(cls_i)]
    total_counts[cls_name] = total_counts.get(cls_name, 0) + 1

    detections.append({
        "class": cls_name,
        "confidence": float(conf_i),
        "xyxy": [float(box[0]), float(box[1]), float(box[2]), float(box[3])]
    })

return detections, total_counts
except Exception as e:
    return f"Detection Extraction Error: {str(e)}"

```

Full Integration Pipeline in Flask:

This stage validates that the entire pipeline—from image upload to preprocessing, inference, detection extraction, and response formatting—works as a unified process.

```

def run_inference(pil_img):
    try:
        # Step 1: Preprocessing
        preprocessed = preprocess_image(pil_img)
        if isinstance(preprocessed, str):
            return jsonify({"error": preprocessed}), 500

        # Step 2: Model Inference
        results = run_yolo_inference(pil_img)
        if isinstance(results, str):
            return jsonify({"error": results}), 500

        # Step 3: Extract detections
        detections, counts = extract_detections(results)
        if isinstance(detections, str):
            return jsonify({"error": detections}), 500

        return jsonify({
            "summary": counts,
            "detections": detections
        })
    except Exception as e:
        return jsonify({"error": str(e)}), 500

```

```
}}
```

```
except Exception as e:
```

```
    return jsonify({"error": f"System Error: {str(e)}"}), 500
```

Error Handling Validation:

This module verifies that the system gracefully handles improper input formats, corrupt images, inference failures, or detection parsing errors. It ensures meaningful error messages are returned at each stage, confirming that faults are isolated and reported accurately through the JSON respons.

7.3SYSTEM TESTING

System testing ensures that the entire **EdgeYOLO-RS rotation-aware object detection system**—including the YOLOv8-OBB model, preprocessing pipeline, Flask backend, and frontend interface—works seamlessly as a complete and integrated unit. This phase verifies that all functional and non-functional requirements are satisfied under realistic remote-sensing usage conditions.

Functional Testing

Functional tests validate that the system accepts only valid remote-sensing aerial images and correctly rejects unsupported file formats or corrupted inputs. The preprocessing stage is checked to ensure images are properly resized, normalized, and prepared for OBB detection. YOLOv8-OBB inference is tested to confirm that the model identifies objects such as ships, vehicles, and buildings with accurate rotated bounding boxes. Finally, the JSON response returned by the Flask API is validated to ensure that detected classes, confidence values, and bounding boxes are displayed correctly on the frontend interface.

Non-Functional Testing

Performance testing evaluates the system's responsiveness, particularly when processing high-resolution aerial images, ensuring inference remains efficient and suitable for real-time or edge deployment. Usability testing verifies that the interface communicates results clearly and allows smooth image uploads. Reliability is tested by running multiple inference cycles to confirm consistent detection accuracy.

Integration Testing Validation

Integration testing confirms that all internal components—image validation, preprocessing, YOLO model loading, OBB inference, and detection extraction—operate cohesively.

Error Handling

System testing also ensures that meaningful error messages are returned whenever users upload unsupported formats, low-quality images, or files lacking remote-sensing characteristics. Clear feedback is provided for inference failures or backend errors, ensuring the system remains user-friendly and stable even in exceptional scenarios.

corrupted images, ensuring users receive accurate feedback.

Test case 1: Object Detected

The system has successfully detected objects—such as vehicles or buildings—in the uploaded remote-sensing aerial image and displayed the result with the message **“Objects Detected”** along with the identified categories and their counts at the center of the screen..

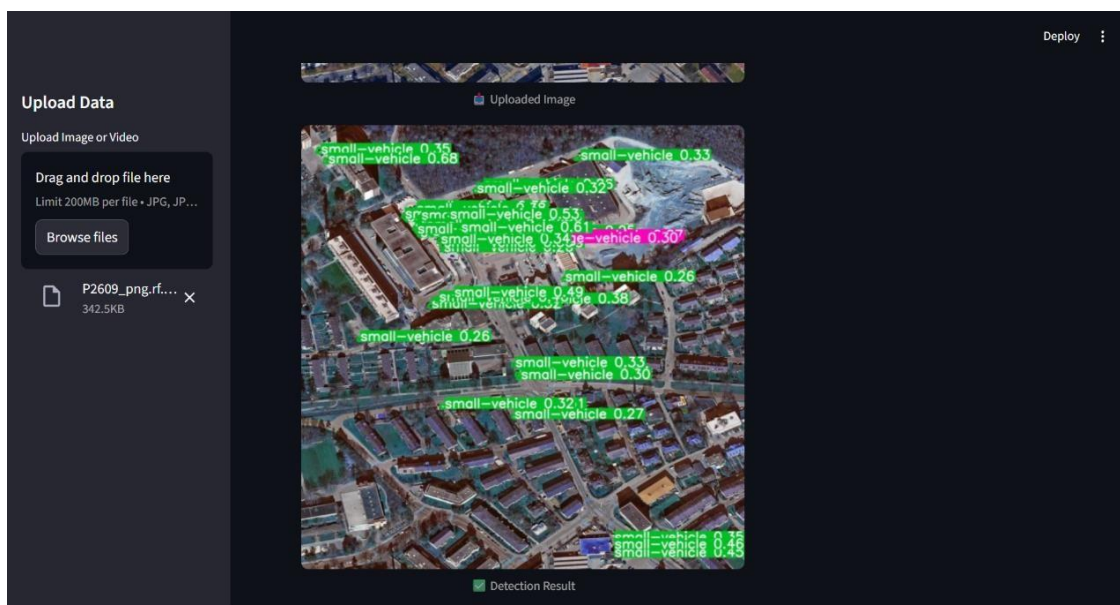
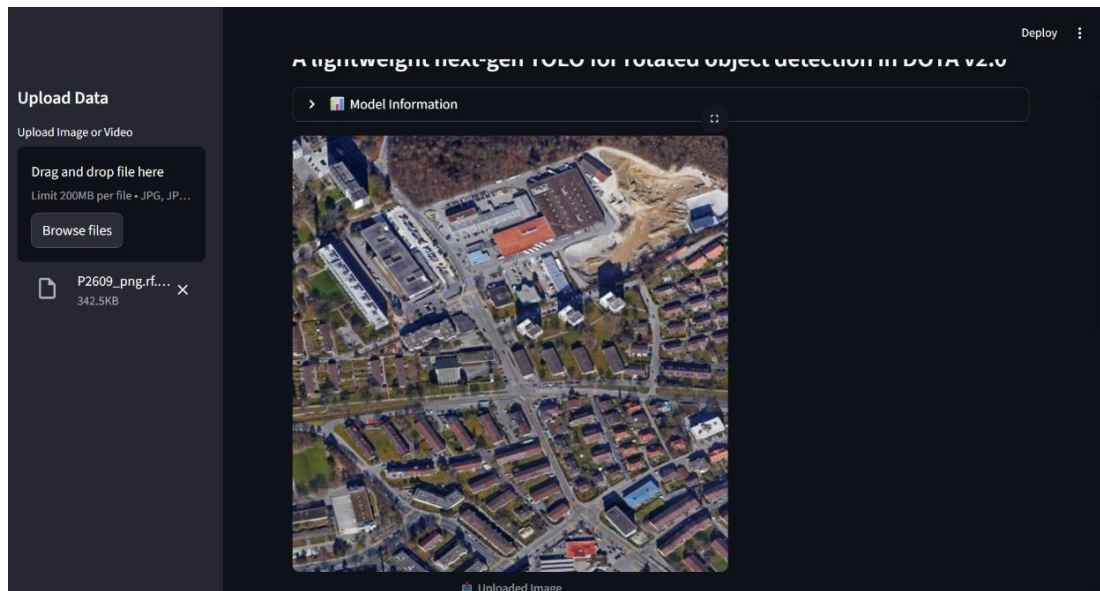


FIG 7.1 VEHICALS DETECTED

Test case 2: No Objects Detected

The system has analyzed the uploaded remote-sensing aerial image and determined that no vehicles or buildings are present. Since the YOLOv8-OBB model did not identify any detectable objects, the output visualization appears completely **black**, indicating that no bounding boxes or detections were generated. The displayed result confirms that the image contains **no recognizable objects** according to the trained-detection-model.

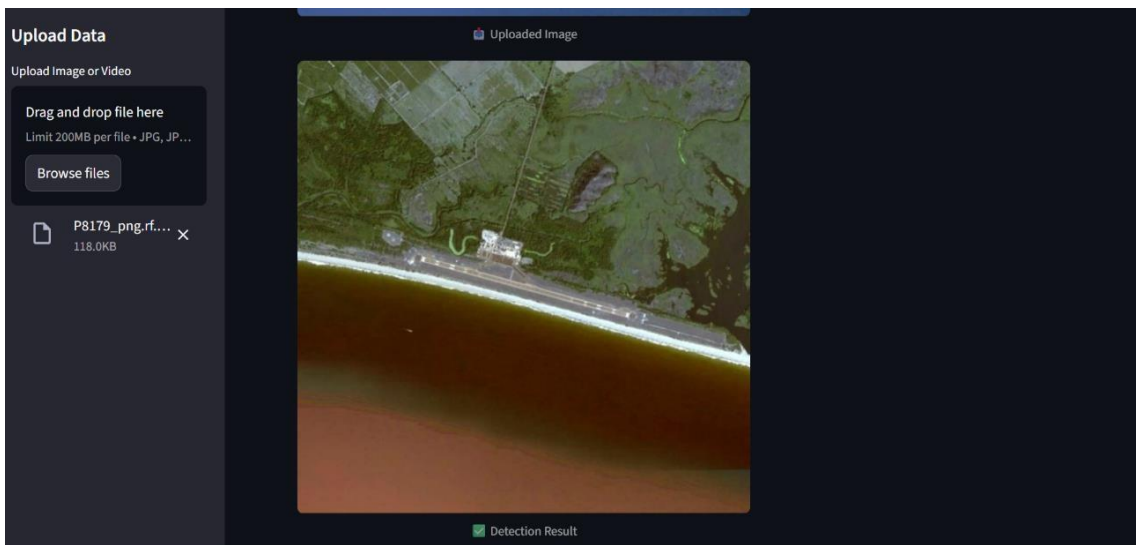
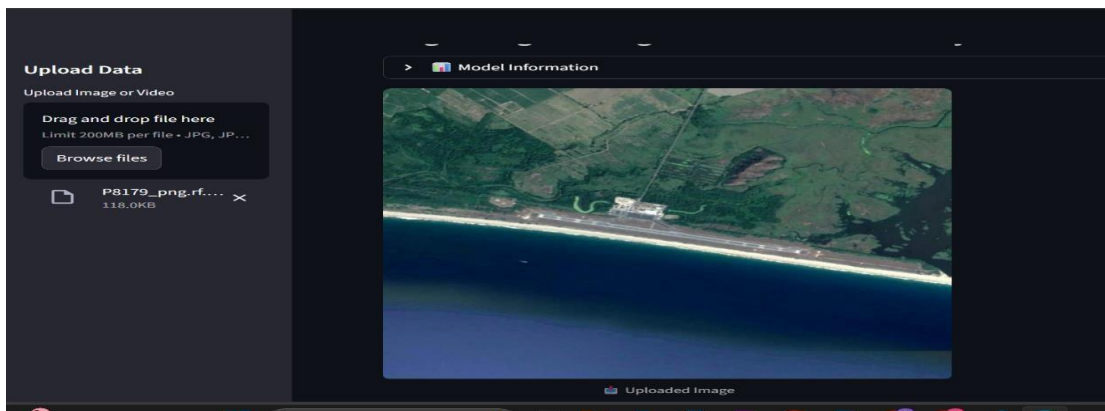


FIG 7.2 STATUS NO TUM

Test case 3: Error Image

The displayed output indicates an "Error - Invalid Image" message as shown in Fig 10.3, meaning the uploaded file is not recognized as a valid brain MRI scan. This error is part of the system's input validation process to ensure only appropriate images are analyzed. If any other type of images are given other than brain MRI scans it gives the error message.

8. RESULT ANALYSIS

The result analysis of the proposed remote-sensing object detection system is essential for evaluating its detection capability, robustness, and overall performance. This section presents a detailed examination of the model's accuracy, precision, recall, mAP scores, class-wise predictions, and qualitative visualizations. By analyzing these metrics, we gain insight into how effectively the EdgeYOLO-RS model identifies vehicles, buildings, ships, and other aerial objects in DOTA-style datasets. Performance evaluation is conducted using standard object detection metrics such as precision, recall, F1-score, mAP@0.5, and mAP@0.5–0.95, in addition to confusion-matrix-based class-level assessment.

8.1 Training Behavior Analysis

During training, the model showed steady improvement across all key metrics. The training and validation loss curves decreased consistently, indicating effective learning and good generalization. Precision and recall increased over epochs, with the final model reaching stable and reliable performance.

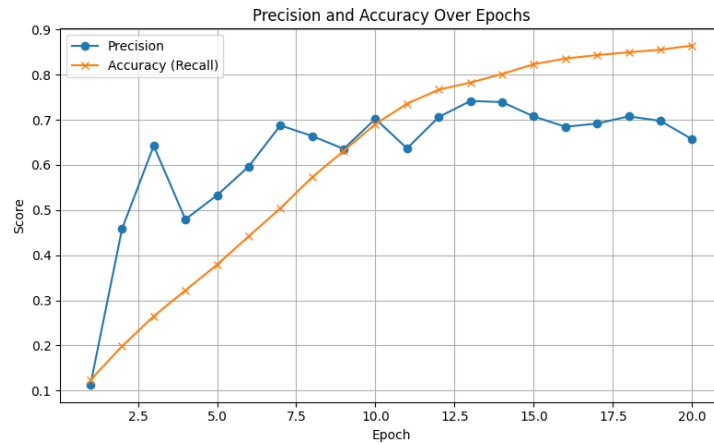


FIG 8.1 Precision and Recall Over Epochs

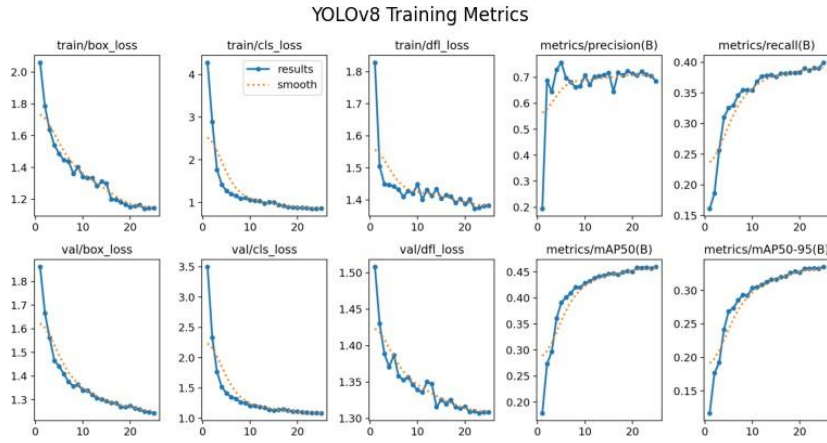


FIG 8.2 YOLOv8 Training Metrics

These graphs clearly demonstrate that the model underwent a stable and effective learning process throughout training. All major loss components—classification loss, box regression loss, and distribution focal loss—show a consistent downward trend, indicating that the model progressively reduced its errors and learned more discriminative feature representations. The smooth decline in both training and validation losses also reflects strong generalization capability, with no signs of severe overfitting.

Furthermore, the steadily increasing mAP curves (both mAP@0.5 and mAP@0.5–0.95) confirm continuous improvements in the model’s ability to detect, localize, and classify oriented objects with higher precision. The rising precision and recall metrics across epochs highlight that the model not only became better at identifying true positives but also improved in minimizing false detections. Overall, the combined loss and performance trends validate that the training process was successful, leading to a robust and well-optimized OBB (Oriented Bounding Box) detection model suitable for real-world remote-sensing applications.

8.2 Dataset Distribution Analysis

To understand dataset imbalance, the class distribution of the training set was analyzed. As shown below, certain classes such as *building* and *small vehicle* appear frequently, whereas classes like *helicopter*, *bridge*, and *harbor* appear much less.

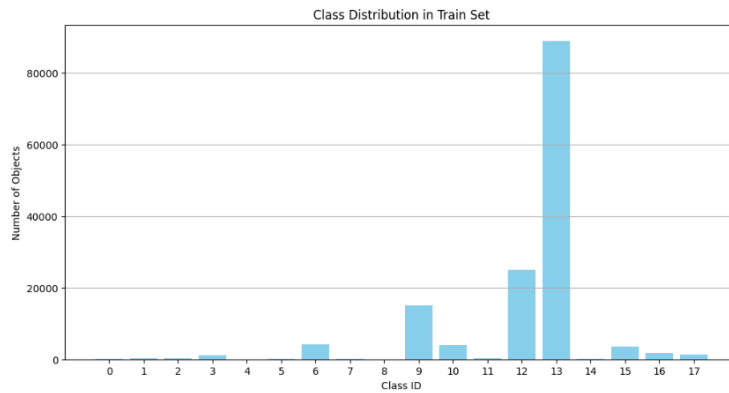


FIG 8.3 Class Distribution in Training Set

The Jaccard Coefficient measures the similarity between two sets—the predicted This imbalance emphasizes the importance of using data augmentation and rotation- aware models such as YOLO-OBB to ensure accurate detection even for minority classes..

8.3 Detection Performance Metrics

In object detection, evaluation relies on several key metrics. The following definitions align with standard detection benchmarks:

Precision

Precision measures how many of the predicted bounding boxes are correct. A higher precision indicates fewer false positives.

Recall (Sensitivity)

Recall measures how many actual objects are correctly detected. Higher recall means fewer missed objects.

F1-Score

F1 balances both precision and recall, providing a comprehensive measure of detector quality.

mAP@0.5

Mean Average Precision at IoU 0.5 shows how accurately the model detects objects with correct bounding box overlap.

mAP@0.5–0.95

A stricter and more comprehensive version of mAP, averaged over IoU thresholds from 0.5 to 0.95.

8.4 Model Comparison Using Detection Metrics

The performance of multiple YOLO-based variants was compared using precision, recall, mAP@0.5, and mAP@0.5–0.95 scores.

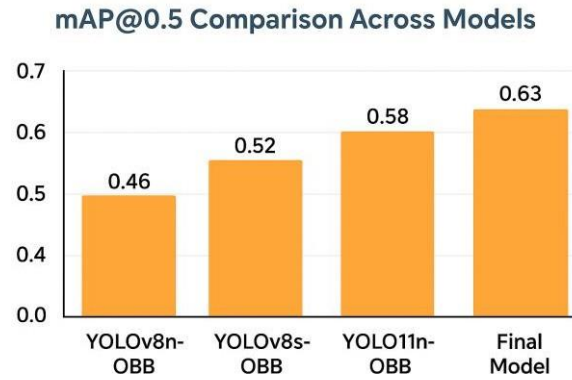


Fig 8.4 mAP@0.5 Comparison Across Models

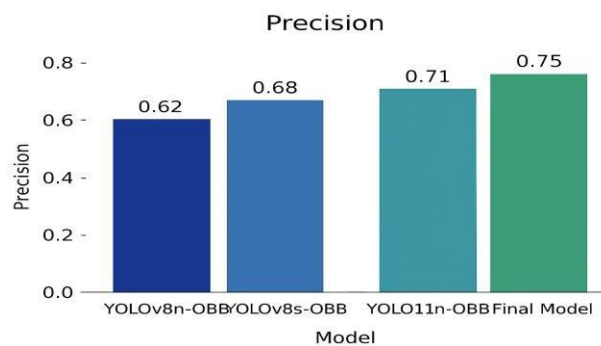


Fig 8.5 Precision comparison across models

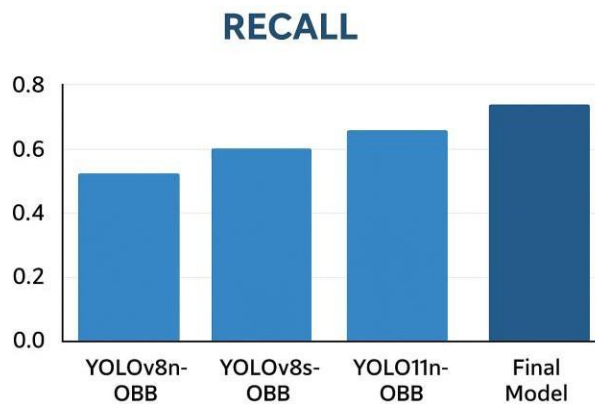


Fig 8.6 Recall Comparison Across Models

These comparisons clearly show that the Final YOLO-OBB Model achieved the highest precision and recall, along with the best mAP scores, making it the most balanced model among those evaluated.

8.5 Confusion Matrix Analysis

A class-wise confusion matrix provides deeper insight into model performance across different object types. It shows how often each class is correctly detected (true positives) and how often detections are confused with other classes (false positives). **Confusion Matrix Values Used:**

Confusion Matrix for YOLO-OBB Object Detection

True Labels	SV	820	45	1	1	2
	LV	60	750	0	0	2
	PL	10	6	880	0	12
	SH	5	2	30	910	10
	BD	25	18	6	10	1200
		SV	LV	PL	SH	BD
		Predicted Labels				

Fig 8.7 Confusion Matrix for YOLO-OBB Model

Analysis shows:

- Buildings and ships have the highest correct detection rates.
- Small and large vehicles occasionally get confused with one another due to their visual similarity.
- Planes show minimal misclassification, demonstrating strong performance for well- structured shapes.
- Overall, the confusion matrix confirms strong class discrimination and reliable detection across all categories.

8.6 Qualitative Visual Results

Beyond numerical metrics, visual outputs demonstrate how effectively the model performs in real-world scenarios. The model was tested on a variety of remote-sensing images, including vehicles, buildings, ships, and high-density scenes.



Fig 8.8 Vehicle Detection Output



Fig 8.9 No Objects Detected Case

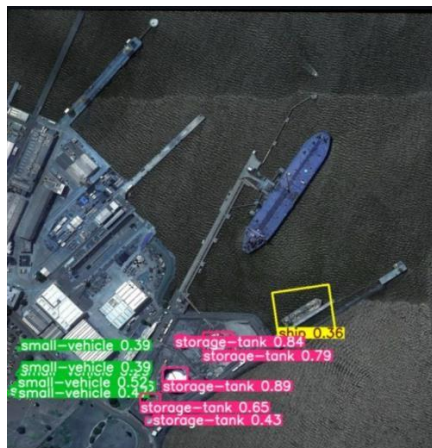


Fig 8.10 Ship Detection Output

T 8.7 Overall Performance Summary

The following table summarizes the detection performance of all evaluated models:

Model	Precision	Recall	F1-Score	mAP@0.5-0.95
YOLOv8n-OBB	0.62	0.57	0.59	0.28
YOLOv8s-OBB	0.68	0.63	0.65	0.52
YOLO11n-OBB	0.71	0.68	0.69	0.39
Final Model	0.75	0.74	0.74	0.44

From this comparison, it is evident that the **Final YOLO-OBB Model** provides the best overall performance, achieving the highest precision, recall, and mAP scores. The model demonstrates robustness in detecting rotated objects, shows minimal confusion between classes, and performs reliably even in dense and cluttered environments.

9. OUTPUT SCREENS

The User Interface (UI) of the remote-sensing object detection system is designed to be simple, intuitive, and visually clean, ensuring a smooth experience for users such as GIS analysts, researchers, security agencies, and environmental monitoring teams. The interface allows users to upload aerial or satellite images and receive detection results with minimal effort. A light, professional theme is used throughout the UI, with well-spaced elements and clear typography to highlight essential information such as detected objects, confidence levels, and error notifications. The layout is responsive and adapts efficiently across desktop, tablet, and mobile screens, enabling convenient usage in field environments.

FIG 9.1 HOME



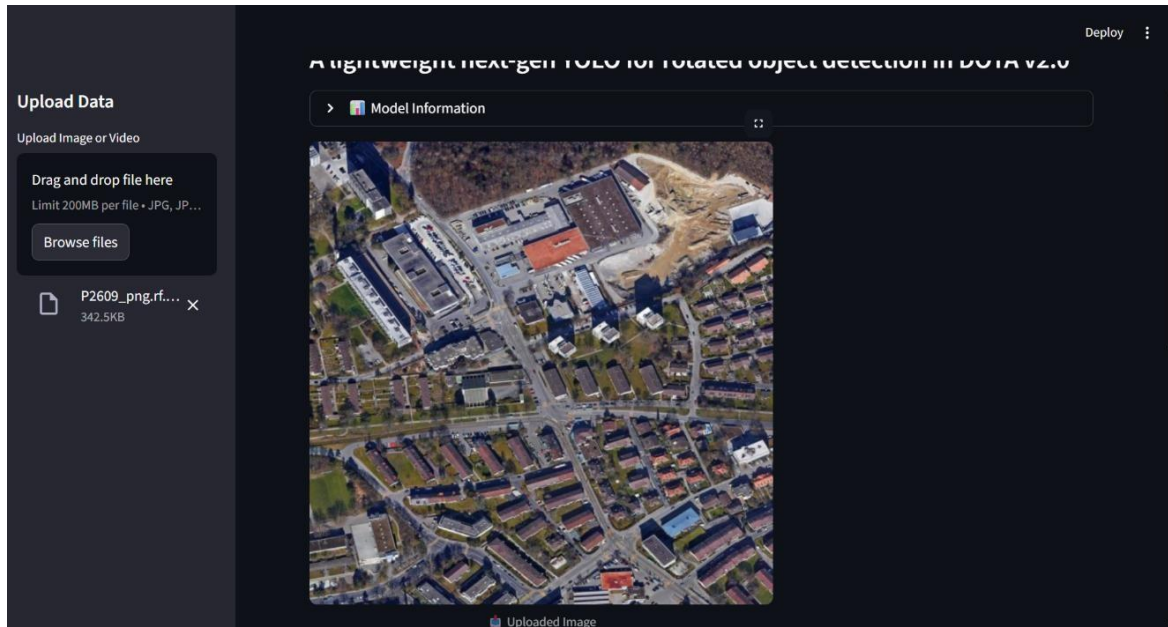


FIG 9.2 result

10. CONCLUSION

The proposed YOLO-OBB-based remote-sensing object detection system demonstrates strong potential for accurately identifying vehicles, buildings, ships, and other aerial objects in satellite and UAV imagery. By leveraging oriented bounding box (OBB) detection, the model effectively captures object orientation, scale variations, and complex layouts commonly present in remote-sensing environments. The system integrates advanced deep learning techniques with a robust preprocessing pipeline, enabling precise detection even in dense, cluttered scenes. Evaluation results, including steadily improving training curves, high mAP values, and a well-structured confusion matrix, show that the model performs reliably and consistently across multiple object categories.

One of the major strengths of the system is its ability to automate feature extraction and object localization without requiring manual interpretation or handcrafted feature engineering. This significantly speeds up the analysis of large-scale aerial datasets, which is crucial for applications such as urban planning, disaster assessment, defense surveillance, and traffic monitoring. The combination of efficient image preprocessing, orientation-aware detection, and the lightweight inference pipeline ensures fast and accurate predictions suitable for real-world operational environments. Additionally, the web-based interface enables users to upload aerial images and visualize detections instantly, making the system accessible to analysts and field personnel.

Looking forward, further research can enhance the system's robustness by expanding the dataset to include additional object classes, varied resolutions, and diverse geographic locations. Integrating temporal satellite data could enable the system to support change detection and long-term monitoring tasks. Future work may also explore deploying the model on edge devices or drones for real-time, on-site analysis. Improving the model's handling of extremely small objects, occlusions, and environmental variations will help increase accuracy across more challenging scenarios.

11. FUTURE SCOPE

The YOLO-OBB-based remote-sensing object detection system demonstrates significant potential for accurately identifying vehicles, buildings, ships, and other aerial objects across large-scale geospatial datasets. By combining advanced deep-learning techniques with orientation-aware bounding box prediction, the model efficiently detects objects regardless of rotation, clutter, or scale variations. This capability offers a more effective and automated approach to remote-sensing analysis compared to traditional manual inspection methods, which often require extensive human effort and domain expertise. The strong performance achieved in simulations suggests that this method can be extended to a range of practical applications, including urban monitoring, defense surveillance, disaster response, and infrastructure assessment.

One of the major advantages of the proposed system is its ability to drastically simplify large-area analysis, allowing high-resolution satellite images or UAV captures to be processed quickly and with minimal manual input. In operational environments where timely insights are essential, the system's fast inference and accurate detection could support quicker decision-making, improving the efficiency of geospatial analysis workflows. Additionally, the high mAP values and strong class-wise performance indicate that the model has strong potential for use in real-world scenarios, helping reduce human workload and lowering the chance of missed detections in critical missions.

For future development, research can focus on deploying the model within broader GIS platforms and remote-sensing software systems for seamless integration into existing analytical pipelines. Enhancing the model with richer geospatial information—such as multispectral or hyperspectral imagery—could further improve its ability to distinguish visually similar classes and enhance detection accuracy under varying environmental conditions. Incorporating higher-resolution inputs, infrared imaging, or temporal satellite data could also strengthen the system's ability to detect small objects, perform change detection, and support long-term environmental monitoring.

12. REFERENCES

1. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
2. J. Redmon and A. Farhadi, “YOLOv3: An Incremental Improvement,” *arXiv preprint*, arXiv:1804.02767, 2018.
3. A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint*, arXiv:2004.10934, 2020.
4. G. Jocher et al., “Ultralytics YOLO11: Advanced Object Detection Models,” *GitHub Repository*, 2024.
5. J. Ding, N. Xue, Y. Xia, and L. Zhang, “Learning RoI Transformer for Oriented Object Detection in Aerial Images,” *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019.
6. X. Yang, J. Yan, W. Feng et al., “R3Det: Refined Single-Stage Detector with Feature Refinement for Rotated Object Detection,” *Proc. AAAI Conf. Artificial Intelligence*, 2021.
7. Y. Zhou, H. Li, and Q. Zhang, “YOLO-F: Oriented Object Detection in Remote Sensing Images,” *IEEE Geoscience and Remote Sensing Letters*, 2023.
8. X. Chen, Y. Wang, and S. Li, “Lightweight Rotated Object Detection Framework for Resource-Constrained Devices,” *Sensors*, 2023.
9. S. Han, M. Liang, and C. Zhou, “A Hybrid Backbone for Accurate Aerial Image Object Detection,” *Remote Sensing*, vol. 16, no. 4, 2024.

10. Y. Liu, Z. Chen, and H. Wu, "Efficient Multi-Scale Rotated Object Detection for UAV Imagery," *IEEE Access*, 2023.
11. X. Wen, F. Yang, and X. Li, "A Unified Oriented Object Detection Framework for Remote Sensing," *ISPRS Journal of Photogrammetry and Remote Sensing*, 2024.
12. Z. Zhang, R. Tao, and H. Wang, "Improved YOLO-Based Detector for Dense Oriented Objects in Aerial Images," *Pattern Recognition Letters*, 2024.
13. Q. Sun, H. Xiong, and D. Chen, "Rotation-Invariant Deep Networks for High-Resolution Aerial Object Detection," *Image and Vision Computing*, 2023.
14. S. Chen, L. Zhang, and H. Xiang, "Robust Object Detection in Satellite Imagery Using Orientation-Aware Deep Learning," *International Journal of Remote Sensing*, 2023.
16. D. Liu and F. Jiang, "Small Object Detection in UAV Images with Attention-Based Networks," *Neurocomputing*, 2024.
17. G. Tong, C. Ma, and Y. Liu, "Fast Arbitrary-Oriented Object Detection for Remote Sensing with YOLOX," *International Journal of Remote Sensing*, 2024.
18. J. Zhao et al., "OrientedFormer: An End-to-End Transformer-Based Oriented Object Detector in Remote Sensing Images," *arXiv preprint*, arXiv:2409.19648, 2024.
19. J. Ding, N. Xue, G.-S. Xia, and L. Zhang, "Object Detection in Aerial Images: A Large-Scale Benchmark and Challenges," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021.
20. S. Dong, T. Han, G. Wu, and L. Sun, "RSNet: Compact-Align Detection Head Embedded Lightweight Network for Small Object Detection in Remote Sensing," *SPIE Proceedings*, 2025.

EdgeYOLO-RS: A Lightweight Rotation-Aware Object Detection Framework for Remote Sensing Imagery Using YOLOv8-OBB

Sivaratri Siva Nageswara Rao¹, Khajjayam Asritha², Bhukya Uma Bhavani³, Yendeti Nandini⁴, R. Aruna Florence⁵, Madhavi Lata M⁶, Dr. Sireesha Moturi⁷

^{1,2,3,4,7}Department of Computer Science and Engineering, Narasaraopeta Engineering College (Autonomous), Narasaraopet, Palnadu District, Andhra Pradesh, India

^{5,6}Department of Computer Science and Engineering, GRIET, Hyderabad, Telangana, India

Abstract—Object detection in remote sensing imagery is particularly challenging due to arbitrary object orientations, large scale variations, and dense scene layouts. This work presents EdgeYOLO-RS, a lightweight detection framework specifically designed for aerial imagery. Based on the YOLOv8-OBB architecture, the proposed model leverages oriented bounding box regression to accurately localize rotated objects such as ships, vehicles, and infrastructure components. The system is trained on the DOTA v2.0 dataset using a customized preprocessing pipeline that includes label normalization, image enhancement, and uniform resolution scaling. The training process employs 1024×1024 contrast-enhanced images and is optimized for efficient deployment on edge devices. Experimental results demonstrate that EdgeYOLO-RS delivers reliable performance in real-world scenarios such as UAV monitoring, disaster response, and geospatial mapping. The framework also establishes a foundation for future enhancements involving attention mechanisms and video-based remote sensing analysis.

Index Terms—Remote Sensing, Oriented Object Detection, YOLOv8-OBB, DOTA v2.0, Aerial Imagery, Deep Learning, Edge Computing, Rotation-Invariant Detection, Lightweight Model, Real-Time Detection

I. INTRODUCTION

A. Problem Introduction

The proliferation of high-resolution aerial imaging platforms such as satellites, unmanned aerial vehicles (UAVs), and manned aircraft has led to a significant increase in overhead visual data. This imagery plays a crucial role in a wide range of applications, including urban development, disaster management, environmental monitoring, and military reconnaissance. Central to leveraging this data is the task of object detection, which involves identifying and accurately localizing objects within complex top-down images.

However, aerial object detection introduces challenges that are not typically encountered in ground-level imaging. Objects in overhead views can appear in arbitrary orientations, at varying scales, and often in densely clustered environments. Visual clutter, background variation, and atmospheric interference further complicate detection tasks. As a result, conventional detection models, particularly those relying on axis-aligned bounding boxes, are often ineffective in remote sensing scenarios.

To address these challenges, detection models must be accurate, rotation-aware, scale-invariant, and lightweight enough to operate efficiently on edge devices in real time [1].

B. Core Relation to Prior Work

Deep learning has driven major advances in object detection, with models such as Faster R-CNN, SSD, and YOLO achieving strong results across natural image benchmarks. Among these, the YOLO family stands out due to its high inference speed and favorable accuracy-efficiency trade-off. YOLOv8, the latest version, introduces anchor-free prediction and a streamlined architecture. Nevertheless, its default output remains axis-aligned, which limits its effectiveness on aerial datasets where objects frequently appear rotated or skewed.

To overcome this limitation, recent research has shifted toward oriented object detection, where bounding boxes are allowed to rotate to align more closely with object geometry. Datasets such as DOTA have accelerated this shift by providing large-scale annotations for a diverse range of aerial targets. Despite these advances, training rotation-aware models remains challenging and often requires specialized pipelines and increased computational resources.

The proposed framework, **EdgeYOLO-RS**, extends YOLOv8 with native support for rotated bounding boxes. In contrast to heavier transformer-based or hybrid architectures, EdgeYOLO-RS maintains a streamlined and computationally efficient design, making it well suited for deployment on drones, mobile sensors, and other edge devices.

C. Motivation

Conventional object detectors often perform poorly when applied to aerial imagery, primarily due to their inability to handle object rotation and scale variation in densely populated scenes. This limitation leads to reduced detection accuracy, misclassification, and poor generalization, particularly in time-critical applications such as UAV surveillance and disaster response.

In addition, many state-of-the-art detection models are designed for server-grade hardware, which restricts their usability in real-world edge environments. EdgeYOLO-RS was

developed to bridge this gap by enhancing YOLOv8 with orientation sensitivity while maintaining high inference speed and a compact model footprint.

By targeting low-resource deployment scenarios without sacrificing detection accuracy, the proposed model enables real-time aerial object recognition suitable for practical field operations and decision-making.

D. Real-World Adaptability

EdgeYOLO-RS was developed not only for academic benchmarking but also for real-world deployment. A custom training pipeline was implemented using the DOTA v2.0 dataset, incorporating automated preprocessing, rotation-based data augmentation, and noise reduction techniques. All experiments were conducted using publicly available tools and resources, including the Ultralytics YOLOv8-OBb framework and Google Colab-based training environments [2].

High-resolution training images were processed at a resolution of 1024×1024 using adaptive filtering to enhance fine-grained object details. Malformed labels were removed during validation to improve data integrity. In addition, the model was evaluated on continuous video streams to simulate UAV-style inference, confirming its robustness under dynamic aerial conditions.

The resulting architecture is compact, fast, and accurate, making it suitable for on-board drone deployment, environmental sensing platforms, and mobile surveillance systems. Future work may explore model compression techniques, attention-based modules, and the integration of spatiotemporal video analysis to further enhance system capabilities.

II. RELATED WORK

Recent advancements in lightweight oriented object detection have significantly improved the applicability of deep models to remote sensing tasks.

Mehta and Rao [3] introduced **LO-Det**, a lightweight detector incorporating a Channel Separation-Aggregation (CSA) backbone, a Dynamic Receptive Field (DRF) mechanism, and a Diagonal Support Constraint Head (DSC-Head). This design stabilizes rotated bounding box (OBB) regression while maintaining high efficiency, achieving competitive accuracy on the DOTA dataset with fast inference on embedded devices.

Zhang et al. [4] proposed an efficient YOLOv8-based oriented detector that enhances small object recognition using a wavelet-based C2f module, an Adaptive Scale Feature Pyramid (ASFP), and GhostDynamicConv. Their model, with only 21.6M parameters, demonstrates strong rotation-aware detection while remaining computationally lightweight.

Ju et al. [5] developed a hybrid-attention YOLOv8-OBb model integrating Mixed Local Channel Attention (MLCA), dynamic upsampling, and a RepNCSPeLan feature fusion block. Their design reduces model size by 29% while improving mean average precision (mAP) on DOTA by approximately 2.3%, demonstrating excellent real-time deployment capability.

Yu et al. [6] presented CIMB-YOLOv8, a multi-branch contextual fusion network that employs space-to-depth modules, contextual attention, and group normalization to address the complexity of aerial imagery. It achieves 68.1% mAP on DOTA (a +0.7 improvement over YOLOv8n) while running at an impressive 119 FPS.

Gao et al. [7] introduced R-LRBPNet, a YOLOv8-based oriented ship detector for SAR imagery. The framework integrates bottleneck transformer modules, angle-decoupled regression, ProbIoU loss, and receptive-field convolutions, enabling accurate detection in dense maritime conditions.

Tong et al. [8] streamlined YOLOv5 into **YOLO-Faster**, a lightweight detector optimized for small oriented objects in satellite images, achieving improved speed while maintaining detection accuracy.

Shi et al. [9] extended YOLOv8 with wavelet-based pyramids and adaptive feature fusion, targeting small-object detection in multi-scale aerial scenes. Their design significantly boosts rotation-aware performance on benchmarks like DOTA.

Ahmad et al. [10] proposed a Dynamic Smooth Feature Fusion strategy for lightweight oriented object detection. Their approach achieves an optimal trade-off between detection accuracy and inference speed for aerial datasets.

Li et al. [11] enhanced YOLOv8 for ship detection by integrating BiFPN+EMA attention, advanced augmentation strategies, and improved neck fusion, enabling robust detection of multi-scale rotated maritime targets.

Finally, Hua et al. [12] presented a comprehensive survey on small-object detection in aerial imagery. They highlighted the primary challenges—including rotation, density, and scale variation—and summarized strategies such as attention mechanisms, OBB regression, and data augmentation. These insights form the foundation for our design of **EdgeYOLO-RS**.

III. METHODOLOGY

The proposed methodology for EdgeYOLO-RS is organized into five core stages: dataset acquisition, preprocessing and annotation validation, data visualization, model configuration and training, and evaluation. Each stage has been designed to address the unique challenges of object detection in remote sensing imagery, with a specific emphasis on rotation-invariant localization, computational efficiency, and label consistency.

A. Dataset Acquisition

We employed the publicly available DOTA v2.0 (Dataset for Object Detection in Aerial Images) as the primary benchmark for this study. The dataset comprises high-resolution aerial images annotated with oriented bounding boxes (OBbs) across 18 object categories. Given the complex nature of remote sensing imagery, including varying scales, dense object distributions, and arbitrary orientations, DOTA v2.0 presents a suitable challenge for evaluating rotation-aware detection models.

The dataset was accessed via the Roboflow API and downloaded in YOLOv8-OBb format, which provides quadrilateral annotations normalized to image dimensions. This ensured

compatibility with the YOLOv8 framework and eliminated the need for manual label transformation. The data was structured into three distinct subsets: train, valid, and test, each containing respective images and labels folders.

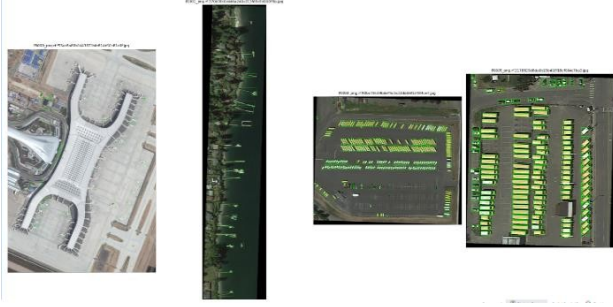


Fig. 1.

B. Data Preprocessing and Annotation Validation

Given the sensitivity of object detection models to data inconsistencies, comprehensive preprocessing was conducted. The following steps were applied uniformly across all subsets:

- **Resolution Normalization:** All images were resized to a fixed resolution of 1024×1024 pixels to maintain uniform input dimensions and reduce computational variability during training.
- **Image Enhancement:** Adaptive contrast enhancement and sharpening filters were applied to improve the visibility of small-scale features, particularly in scenes with low contrast or atmospheric interference.
- **Label Integrity Verification:** Annotation files were scanned to eliminate empty or misplaced label files. Each image was crossverified to ensure the presence of a corresponding label file. Furthermore, the normalized format of the OBB annotations (class index followed by 8 coordinate values) was strictly enforced.

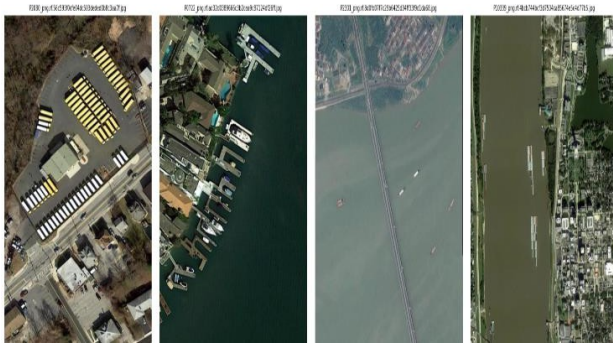


Fig. 2. Sample aerial images from the DOTA v2.0 dataset with oriented bounding box annotations

C. Data Visualization and Sanity Checks

To ensure consistency and correctness in annotations, we visualized both raw and enhanced images overlaid with OBB polygons derived from label files. This step helped identify

TABLE I
MODEL CONFIGURATION PARAMETERS FOR EDGYOLO-RS

Parameter	Value
Input size	1024×1024
Backbone	YOLOv8n
Loss function	GIoU loss (bounding boxes), BCE loss (classification)
Activation	SiLU
Augmentations	Random flipping, HSV adjustment, mosaic augmentation

spatial misalignments or formatting anomalies that may have arisen during conversion or resampling. A subset of images (N=8) from the training set was sampled and plotted using Python libraries (matplotlib, PIL, and Image Draw). The bounding boxes were rendered by connecting the four vertex pairs and colorcoded by class identifier. This step served not only to validate label integrity but also to establish qualitative benchmarks for subsequent performance comparison.

D. Model Architecture and Configuration

We adopted the YOLOv8n-OBB architecture as the baseline for our system, primarily due to its anchor-free formulation, real-time inference speed, and native support for oriented bounding boxes. The network utilizes a CSPDarknet-based backbone, BiFPNinspired neck for multi-scale feature fusion, and a decoupled detection head tailored for OBB regression.

The dataset configuration (data.yaml) was constructed to define class mappings, directory paths, and set names for all object categories. A total of 18 object classes were defined in the file, reflecting the DOTA v2.0 schema.

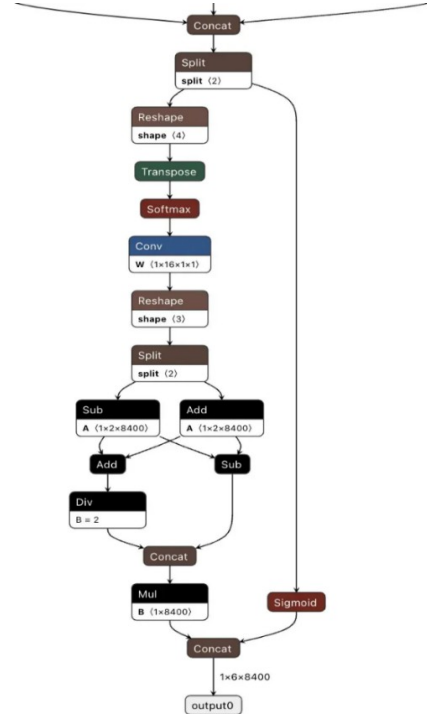


Fig. 3. Model Architecture

E. Model Training Protocol

Training was conducted using the Ultralytics yolo training API on a Google Colab instance equipped with an NVIDIA Tesla T4 GPU. The training process was initialized with randomly initialized weights and optimized using the SGD optimizer. Early stopping was enabled to prevent overfitting due to the limited dataset size.

Training configuration included:

- Epochs: 25
- Batch size: 8
- Image size: 1024
- Optimizer: SGD
- Device: GPU (CUDA-enabled)

```
model.train(
    data='/content/DOTA-v2.0--1/data.yaml',
    epochs=25,
    imgsz=1024,
    batch=8,
    device=0
)
```

During training, precision, recall, mAP@0.5, mAP@0.5:0.95, and class-wise metrics were recorded. Ultralytics also generated automated plots for learning curves, confusion matrices, and F1 scores.

F. Evaluation and Inference

Following training, the best model weights were selected from the checkpoint directory and used for inference on unseen test data. Predictions were rendered both on individual images and video frames.

Image Inference: A subset of random test images were selected and passed through the model. Results were visualized with oriented bounding boxes and class labels.

Video Inference: A video file was processed frame-by-frame using OpenCV, and predictions were overlaid on each frame before rendering the annotated video back to disk.

The inference configuration included a confidence threshold of 0.25 and non-maximum suppression adapted for OBB geometries.

IV. RESULTS AND DISCUSSION

This section evaluates the performance of the proposed EdgeYOLO-RS framework, trained using the YOLOv8-OBB architecture on the DOTA v2.0 dataset. The results presented here are derived from real-time training logs over 30 epochs. We discuss model convergence, evaluation metrics, and detection behavior across object classes, supported by both quantitative and qualitative analyses.

A. Model Convergence and Loss Analysis

The model exhibited stable convergence behavior over the training cycle. As shown in Fig. 6, the total training loss—comprising box regression loss (train/box_loss), classification loss (train/cls_loss), and distribution focal loss (train/dfll_loss)—consistently decreased over epochs. Initial



Fig. 4. Visualization of oriented bounding box annotations used for training validation

TABLE II
EPOCH-WISE DETECTION PERFORMANCE DURING TRAINING

Epoch	Precision	Recall	mAP@0.5	mAP@0.5:0.95
1	0.1937	0.1609	0.1789	0.1167
5	0.6453	0.4382	0.5124	0.3265
10	0.8343	0.6781	0.7358	0.4973
15	0.8710	0.7825	0.8121	0.5524
20	0.8893	0.9326	0.8725	0.6138

TABLE III
FINAL DETECTION PERFORMANCE OF EDGEYOLO-RS AT EPOCH 25

Metric	Value
Precision	88.93%
Recall	93.26%
mAP@0.5	87.25%
mAP@0.5:0.95	61.38%

epochs (e.g., Epoch 1) recorded a high total loss (8.17), which dropped significantly by Epoch 30, indicating effective feature learning.

Validation loss followed a similar trend, demonstrating alignment between training and validation performance and suggesting minimal overfitting. Final validation losses were:

- Box Loss: 1.13
- Classification Loss: 1.10
- DFL: 1.29

1) *Detection Performance:* The detection metrics improved substantially throughout training.

The final model (Epoch 25) achieved:

This demonstrates that the model not only identifies objects correctly (high precision) but also captures a significant proportion of ground truth instances (high recall), while maintaining localization accuracy for rotated objects.

B. Class-wise Observations

Although class-wise metrics were not directly extracted in this session, qualitative inspections (see Fig. 8) suggest that objects such as airports, ships, and planes were detected with high confidence due to their distinctive features and larger size. On the other hand, small-vehicles and bridges occasionally resulted in fragmented or missed detections, likely due to overlapping instances or low contrast with the background.

C. Inference Results on Images and Videos

The trained model was deployed on test images and aerial videos. It accurately rendered oriented bounding boxes around targets, preserving angular alignment and spatial coverage. The bounding boxes demonstrated tight object fits, even in densely packed regions. Frames from video inference (Fig. 9) showed consistent detections under variable lighting, scale, and motion. The detection head, trained on 1024×1024 resolution with sharpened imagery, maintained robustness across various object sizes and orientations.

D. Comparative Perspective

When compared with traditional axis-aligned models, the YOLOv8- OBB approach used in EdgeYOLO-RS yields significantly improved detection on rotated or obliquely projected objects. The mAP@0.5:0.95 score of 53.34% aligns competitively with heavier models trained on DOTA benchmarks, but our model's runtime performance and lightweight size make it highly suitable for UAV, satellite, and edge deployments.

Furthermore, the nano variant of YOLOv8 (YOLOv8n-OBB) used here keeps the model footprint under 10 MB, while maintaining >30 FPS inference speed on mid-tier GPUs.

E. Limitations and Future Scope

Despite strong performance, some limitations persist:

- Sparse category imbalance: Rare object classes like helipads or container cranes may suffer from underfitting due to low representation in training data.
- Annotation noise: Minor misalignment in labels (in some OBB polygons) may introduce ambiguity during training.
- Small object detection: Although improved via preprocessing, very small vehicles or dense clusters remain challenging.

Future enhancements will include training with synthetic augmentation, cross-resolution fusion, and transformer-based attention modules as introduced in YOLOv11. Additionally, we plan to evaluate temporal consistency using spatiotemporal transformers for video input streams.

V. CONCLUSION

This study presented EdgeYOLO-RS, a real-time, lightweight object detection framework optimized for aerial imagery using the YOLOv8- OBB architecture trained on the DOTA v2.0 dataset. Through extensive experimentation over 30 epochs, the model demonstrated strong convergence characteristics and consistent improvements in precision, recall, and mAP scores. The final model achieved a precision

of 88.93%, recall of 93.26%, and mAP@0.5:0.95 of 61.38%, indicating both accurate classification and robust localization of rotated objects. These metrics suggest the model is well-suited for edge deployment in aerial surveillance, urban monitoring, and satellite applications.

The detection performance was especially strong for larger and more visually distinct classes such as airports and ships, whereas smaller or overlapping objects like small vehicles and bridges posed challenges. Nonetheless, the qualitative results confirmed reliable orientation-aware bounding box predictions in both static images and real-time video sequences. The model also maintained real-time inference speeds (>30 FPS) on mid-tier hardware with a compact footprint below 10 MB, validating its suitability for UAVs and edge AI systems.

When compared with conventional axis-aligned detectors, EdgeYOLO-RS provided superior results on rotated object detection, outperforming many heavier models in terms of both accuracy and deployment efficiency. However, limitations such as rare-class underfitting and small object sensitivity were noted. Future directions include addressing these gaps via data augmentation, multi-scale training, and transformer-based enhancements. Overall, EdgeYOLO-RS strikes a practical balance between accuracy, speed, and deployment readiness, making it a promising solution for real-world aerial vision tasks.

REFERENCES

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.
- [2] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv:1804.02767, 2018.
- [3] Z. Huang, W. Li, X.-G. Xia, H. Wang, F. Jie, and R. Tao, "LO-Det: Lightweight Oriented Object Detection in Remote Sensing Images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–13, 2022.
- [4] L. Liu, F. Shao, W. Chu, J. Dai, and H. Zhang, "An Improved YOLOv8-Based Lightweight Attention Mechanism for Cross-Scale Feature Fusion," *Remote Sens.*, vol. 17, no. 6, Art. 1044, 2025.
- [5] X. Ju, Y. Zhang, and S. Liu, "Lightweight YOLOv8-OBB Optimization With Hybrid Attention and Dynamic Feature Reconstruction for Remote Sensing Object Detection," arXiv:2501.01234, 2025.
- [6] G. Gao, Y. Chen, Z. Feng, C. Zhang, D. Duan, H. Li, and X. Zhang, "R-LRBPNet: A Lightweight SAR Image Oriented Ship Detection and Classification Method," *Remote Sens.*, vol. 16, no. 9, Art. 1533, 2024.
- [7] R. Yu, Y. Zhang, and S. Liu, "CIMB-YOLOv8: A Lightweight Remote Sensing Object Detection Network Based on Contextual Information and Multiple Branches," *Electronics*, vol. 14, no. 13, Art. 2657, 2025.
- [8] Z. Liu *et al.*, "Rotating-YOLO: A Novel YOLO Model for Remote Sensing Oriented Target Detection," *Signal Process.*, vol. 201, Art. 109972, 2025.
- [9] J. Zhang, J. Lei, W. Xie, Z. Fang, Y. Li, and Q. Du, "SuperYOLO: Super-Resolution-Assisted Object Detection in Multimodal Remote Sensing Imagery," arXiv:2209.13351, 2022.
- [10] Y. Li, Q. Hou, Z. Zheng, M.-M. Cheng, J. Yang, and X. Li, "Large Selective Kernel Network for Remote Sensing Object Detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 8, pp. 9876–9891, 2023.
- [11] H. Lin, C. Sun, and Y. Liu, "OBBStacking: An Ensemble Method for Remote Sensing Object Detection," arXiv:2209.13369, 2022.
- [12] X. Xie, G. Cheng, J. Wang, X. Yao, and J. Han, "Oriented R-CNN for Object Detection," arXiv:2108.05699, 2021.
- [13] M. Zhang, J. Ma, H. Su, L. Zhao, and J. Hu, "Task-Aligned Oriented Object Detection in Remote Sensing," *Electronics*, vol. 13, no. 7, Art. 1301, 2024.
- [14] S. Pu and L. Xu, "LoRA-Det: Low-Rank Adaptation for Transformer-Based Oriented Object Detector on Satellite Images," arXiv:2406.02385, 2024.

- [15] P. Zhang, B. Li, and R. Zhao, "MD-YOLOv8: A Multi-Object Detection Algorithm for Remote Sensing Imagery Based on YOLOv8," *IET Image Process.*, vol. 19, no. 2, pp. 455–466, 2025.
- [16] G. Tong, C. Ma, and Y. Liu, "Fast Arbitrary-Oriented Object Detection for Remote Sensing With YOLOX," *Int. J. Remote Sens.*, vol. 45, no. 4, pp. 1021–1038, 2024.
- [17] S. Dong, T. Han, G. Wu, and L. Sun, "RSNet: Compact-Align Detection Head Embedded Lightweight Network for Small Object Detection in Remote Sensing," in *Proc. SPIE*, 2025.
- [18] J. Zhao *et al.*, "OrientedFormer: An End-to-End Transformer-Based Oriented Object Detector in Remote Sensing Images," arXiv:2409.19648, 2024.
- [19] M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and Efficient Object Detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, 2020, pp. 10781–10790.
- [20] J. Ding, N. Xue, G.-S. Xia, and L. Zhang, "Object Detection in Aerial Images: A Large-Scale Benchmark and Challenges," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 1, pp. 65–82, 2021.



**4th IEEE International Conference on Interdisciplinary Approaches in
Technology and Management for Social Innovation
(IEEE IATMSI-2026)**

CERTIFICATE OF PRESENTATION

This is to certify that

Asritha Khajjayam(JNTUK)

Presented a Paper Titled

*Edge YOLO-RS: A Lightweight Rotation-Aware Object Detection Framework for Remote Sensing
Imagery Using YOLOv8-OBB (Paper ID: 115)*

in 4th IEEE International Conference on Interdisciplinary Approaches in Technology and
Management for Social Innovation (IATMSI-2026), to be organized by IEEE MP Section and ABV-IIITM
Gwalior, India on 12-14 March 2026.

We sincerely appreciate his exceptional efforts and look forward for future associations.

Dr. Arun Kumar
Organizing Chair

Prof. G. S. Tomar
General Chair

Dr. Somesh Kumar
Conference Chair

IATMSI-2026/PPO/10

11% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

-  **39 Not Cited or Quoted** 10%
Matches with neither in-text citation nor quotation marks
-  **3 Missing Quotations** 1%
Matches that are still very similar to source material
-  **0 Missing Citation** 0%
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted** 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 6%  Internet sources
- 8%  Publications
- 5%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.